

Raytracer Mathematics of light & geometry

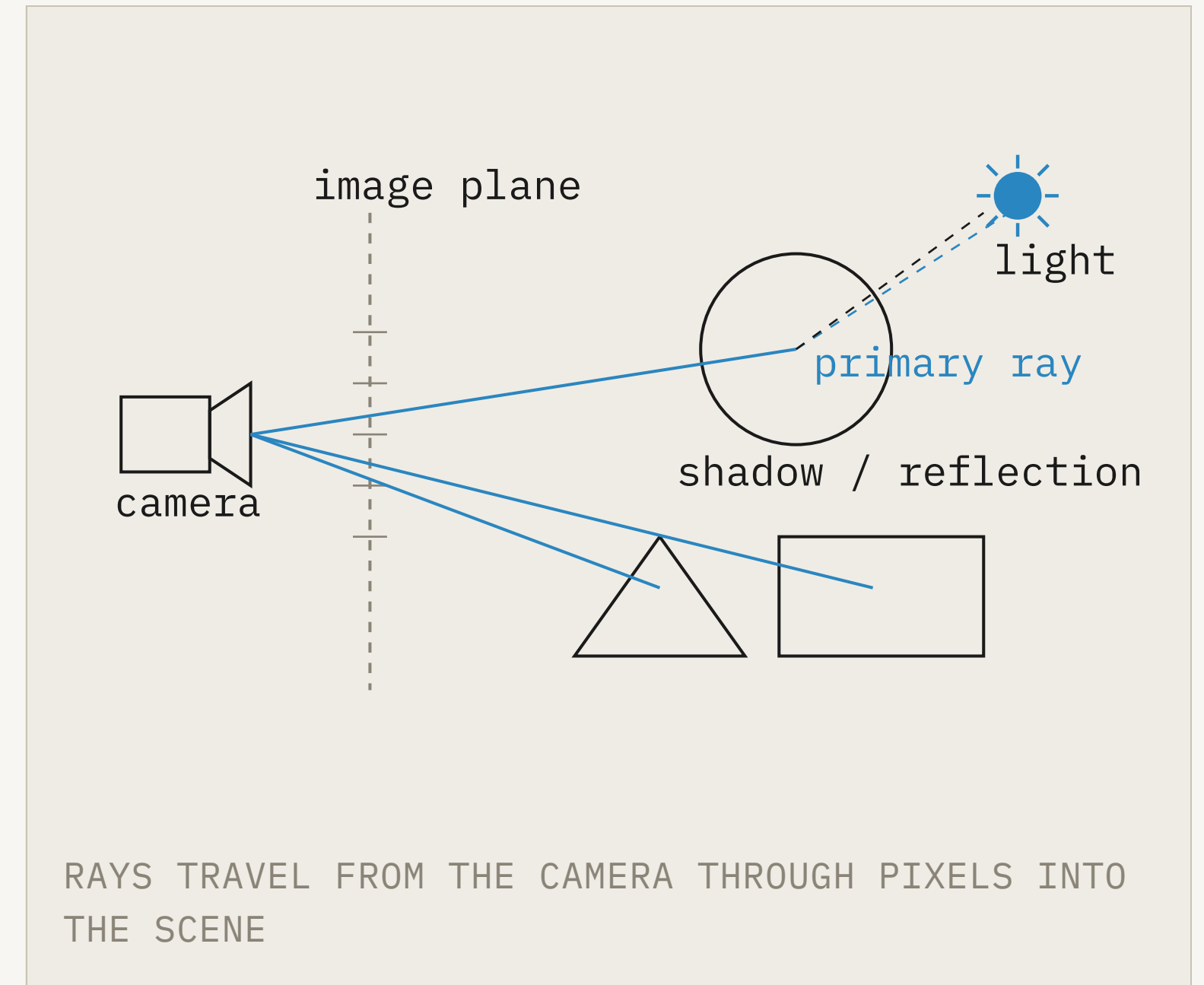
A walkthrough of the equations powering every ray, every surface, every shading and acceleration decision in the renderer.

02 What is a raytracer?

DEFINITION

A raytracer is a rendering algorithm that synthesises an image by simulating the propagation of light through a virtual scene, one ray at a time.

- Each pixel of the image is the result of one or several rays cast from the camera into the 3D scene.
- For every ray, we compute the closest intersection with the scene, evaluate the local material, and gather light contributions.
- Effects such as shadows, reflections, refraction and ambient occlusion emerge naturally from the same recursive model.



USED IN
Film VFX

USED IN
Product & arch.
viz

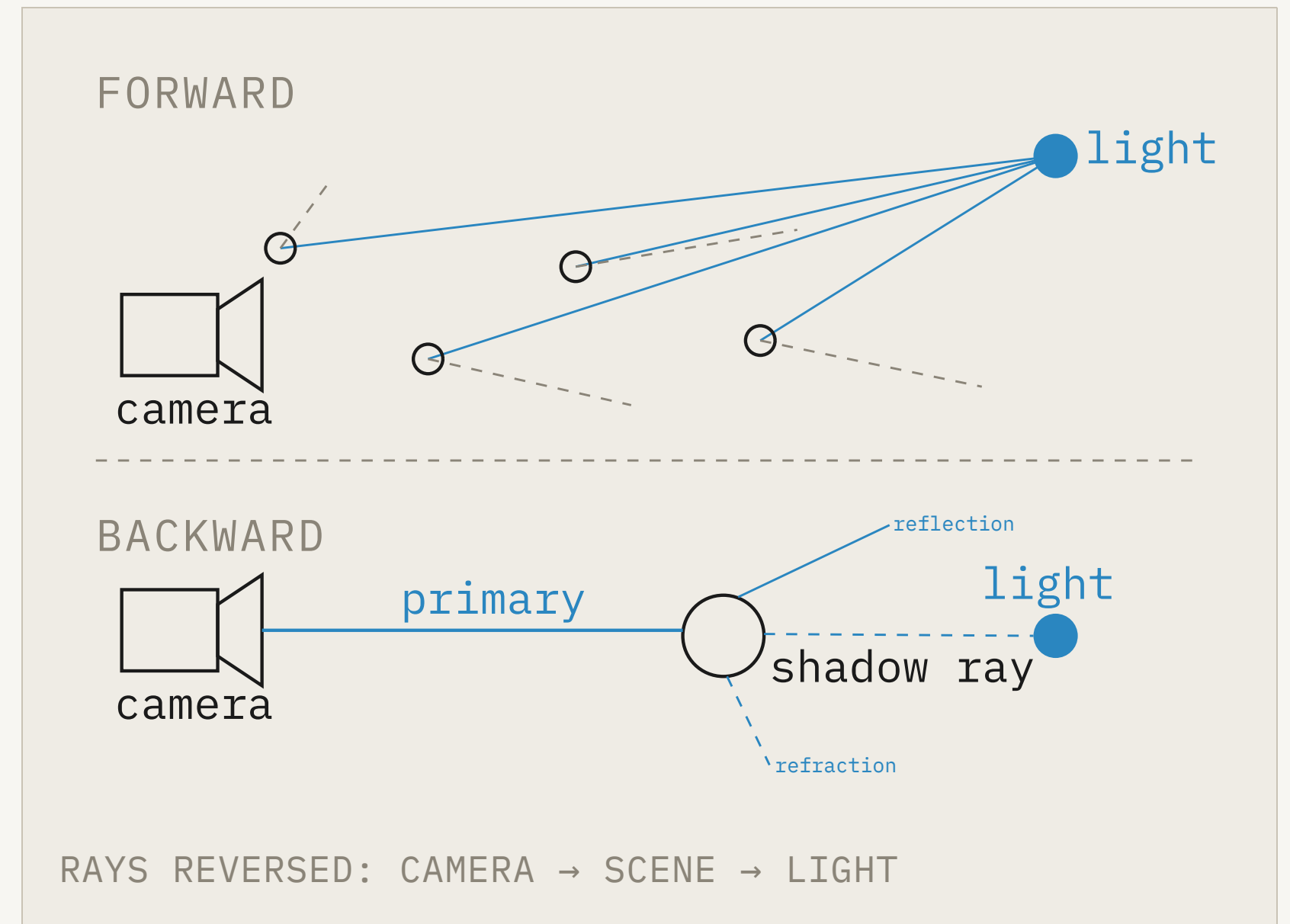
USED IN
Scientific
imaging

03 Backward ray tracing

PRINCIPLE

Physically, light flows from sources to the eye. The raytracer reverses this: it follows rays from the camera outwards, because only those reaching the camera contribute to the image.

- Forward (physical): photons leave the light, scatter through the scene, only a tiny fraction reach the camera. Wasteful.
- Backward (used here): rays leave the camera through every pixel. Exactly one path per pixel needs to be evaluated.
- The recursive nature of the algorithm reproduces secondary effects (reflections, refractions) by spawning new rays at every hit.



04 The rendering equation

KAJIYA, 1986

All physically-based renderers approximate the same integral: outgoing radiance at a point is the emitted light plus the integral of all incoming light, weighted by the surface's BRDF and Lambert's cosine.

$$L_o(p, \omega_o) = L_e(p, \omega_o) + \int_{\Omega} f_r L_i(\omega_i \cdot \mathbf{n}) d\omega_i$$

$L_o(p, \omega_o)$	Outgoing radiance at p in direction ω_o
L_e	Light emitted by the surface itself
f_r	BRDF — fraction of ω_i reflected into ω_o
$L_i(p, \omega_i)$	Incoming radiance from direction ω_i
$(\omega_i \cdot \mathbf{n})$	Lambert cosine term over the hemisphere Ω

This raytracer approximates the integral with three discrete contributions:

- Direct lighting — one shadow ray per light source.
- Indirect lighting — a single recursive bounce (reflection, refraction or transparency).
- Ambient occlusion — Monte-Carlo sampling of the upper hemisphere.

05 The traceRay algorithm

RECURSIVE CORE

```
function traceRay(ray, scene, depth):
    hit = closestHit(ray, scene)           // BVH lookup
    if no hit: return backgroundColor

    s = hit.material.scatter(ray, hit)
    n = s.modifiedNormal ?? hit.normal

    color = (0, 0, 0)
    for light in scene.lights:
        sample = light.getSample(hit.point, n)
        if not shadowed(hit.point, sample):
            color += lambert(n, sample) * s.attenuation
            color += phong(s, sample, ray)

    if depth > 0 and s.secondaryRay:
        indirect = traceRay(s.secondaryRay, scene, depth-1)
        color += s.attenuation * indirect

    return color
```

Five stages

- ① Intersection. Find the nearest surface hit via the BVH.
- ② Scatter. Material returns an attenuation colour and an optional secondary ray.
- ③ Direct lighting. For each light, send a shadow ray and accumulate diffuse + specular.
- ④ Recursion. If a secondary ray was produced, trace it (up to MAX_DEPTH).
- ⑤ Combine. Return the sum of direct and indirect contributions.

MAX_DEPTH	RAYBIAS	E
10	10^{-4}	10^{-12}

06 Ray generation

A ray is the parametric line cast from the camera origin $\mathbf{O}$ along a normalised direction $\mathbf{d}$.

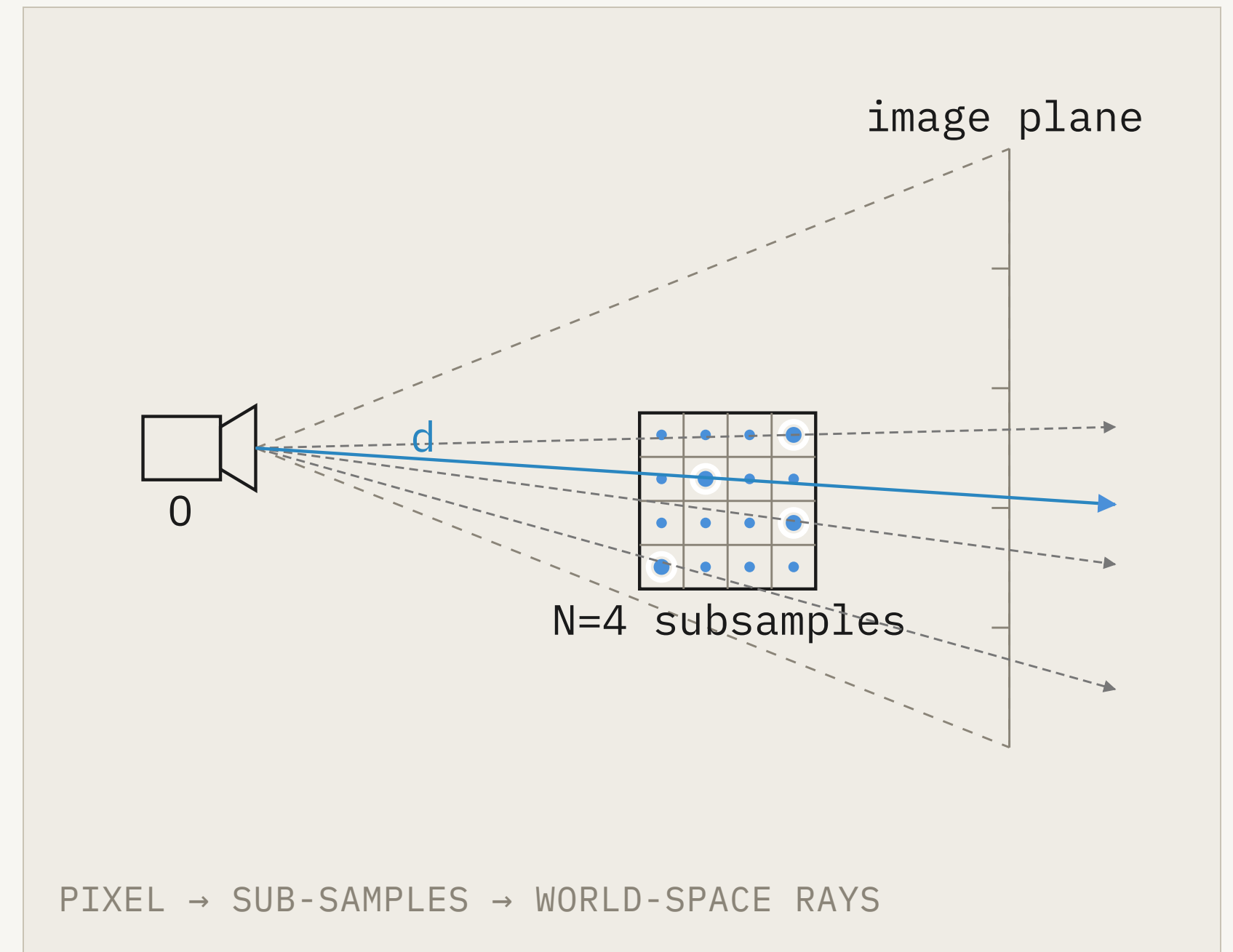
$$P(t) = \mathbf{O} + t\mathbf{d}, \quad t > 0, \quad \|\mathbf{d}\| = 1$$

For each pixel (x, y) , the camera maps NDC coordinates to a direction inside the view frustum, then rotates by the camera's Euler matrix to world space. The vertical FOV and aspect ratio set the frustum shape.

N×N antialiasing

$$u = x + \frac{i+0.5}{N}, \quad v = y + \frac{j+0.5}{N}, \quad i, j \in [0, N)$$

N^2 stratified sub-pixel samples are averaged to produce the final pixel colour.



Primitives. How rays meet geometry.

08 Sphere

PRIMITIVE 1 / 9 • QUADRATIC

The set of points at distance r from a center C .
Intersection reduces to a scalar quadratic in t .

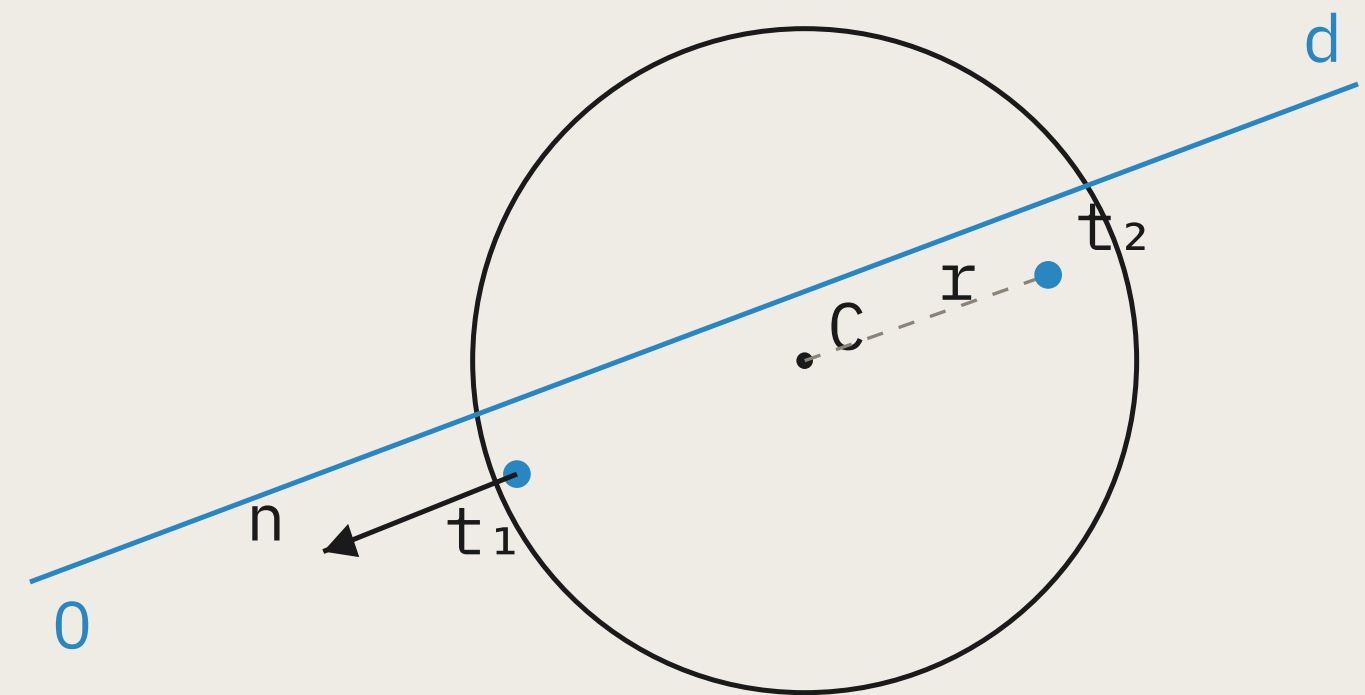
$$\|\mathbf{O} + t\mathbf{d} - \mathbf{C}\|^2 = r^2$$

$$at^2 + 2ht + c = 0$$

$$a = \mathbf{d} \cdot \mathbf{d}, \quad h = \mathbf{d} \cdot \overrightarrow{CO}, \quad c = \|\overrightarrow{CO}\|^2 - r^2$$

$$\Delta = h^2 - ac, \quad t = \frac{-h \pm \sqrt{\Delta}}{a}$$

Normal: $\mathbf{n} = \frac{\mathbf{P} - \mathbf{C}}{r}$ · UV: $u = \frac{1}{2} + \frac{\text{atan2}(n_z, n_x)}{2\pi}$, $v = \frac{1}{2} - \frac{\arcsin n_y}{\pi}$



TWO REAL ROOTS → TWO HITS; TAKE THE SMALLEST POSITIVE

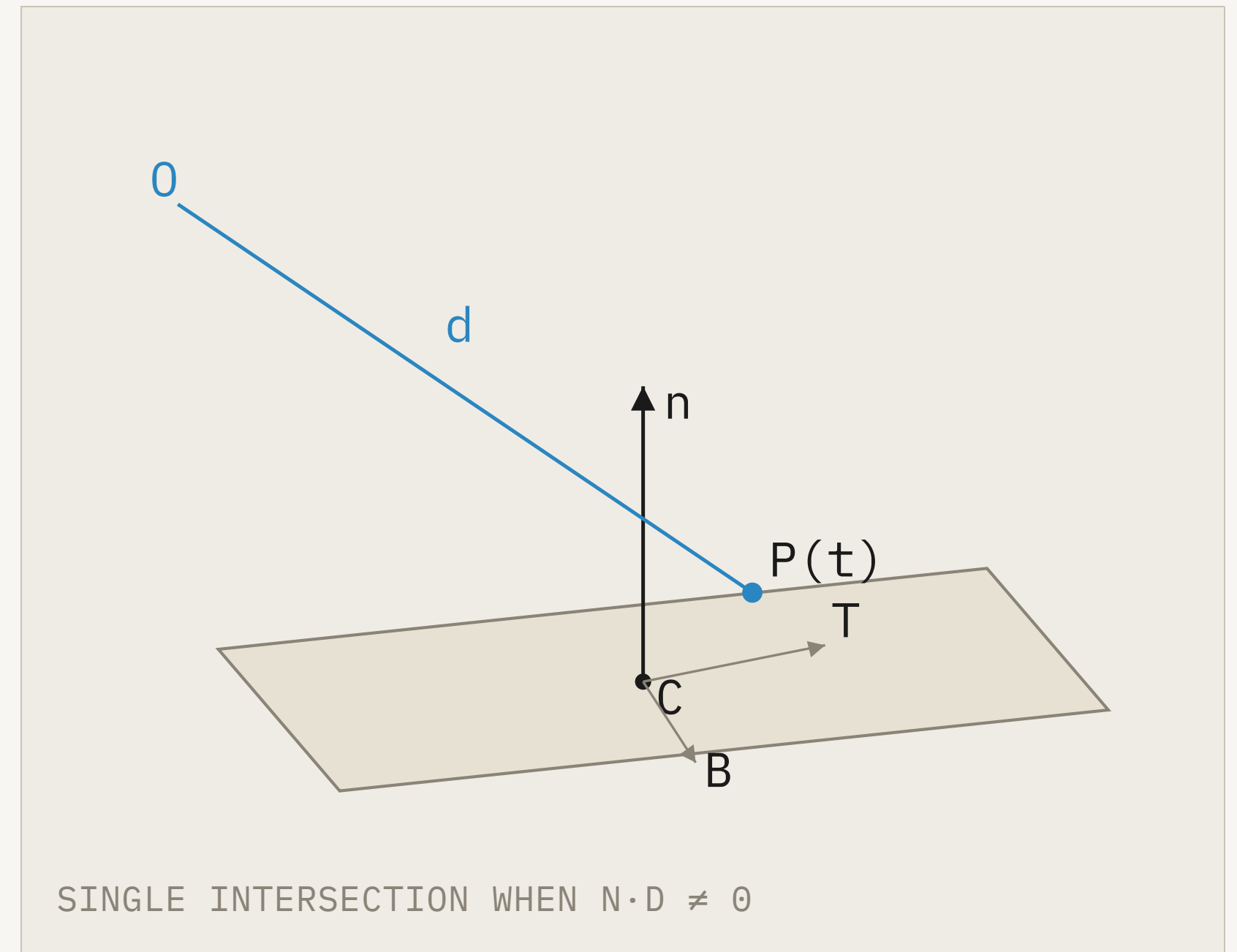
A plane is fully determined by a point C and a unit normal $\mathbf{n}$. Intersection is a single scalar division.

$$\mathbf{n} \cdot (P - \mathbf{C}) = 0$$

$$\mathbf{n} \cdot (\mathbf{O} + t\mathbf{d} - \mathbf{C}) = 0 \implies t = \frac{\mathbf{n} \cdot (\mathbf{C} - \mathbf{O})}{\mathbf{n} \cdot \mathbf{d}}$$

- $\mathbf{n} \cdot \mathbf{d} \approx 0 \rightarrow$ ray is parallel to the plane, no intersection.
- Otherwise accept the hit only when $t > \varepsilon$.
- UV coordinates use a tangent frame (T, B) built from $\mathbf{n}$ via Gram-Schmidt.

Normal: always $\pm \mathbf{n}$, flipped by the `frontFace` flag.



10 Cylinder

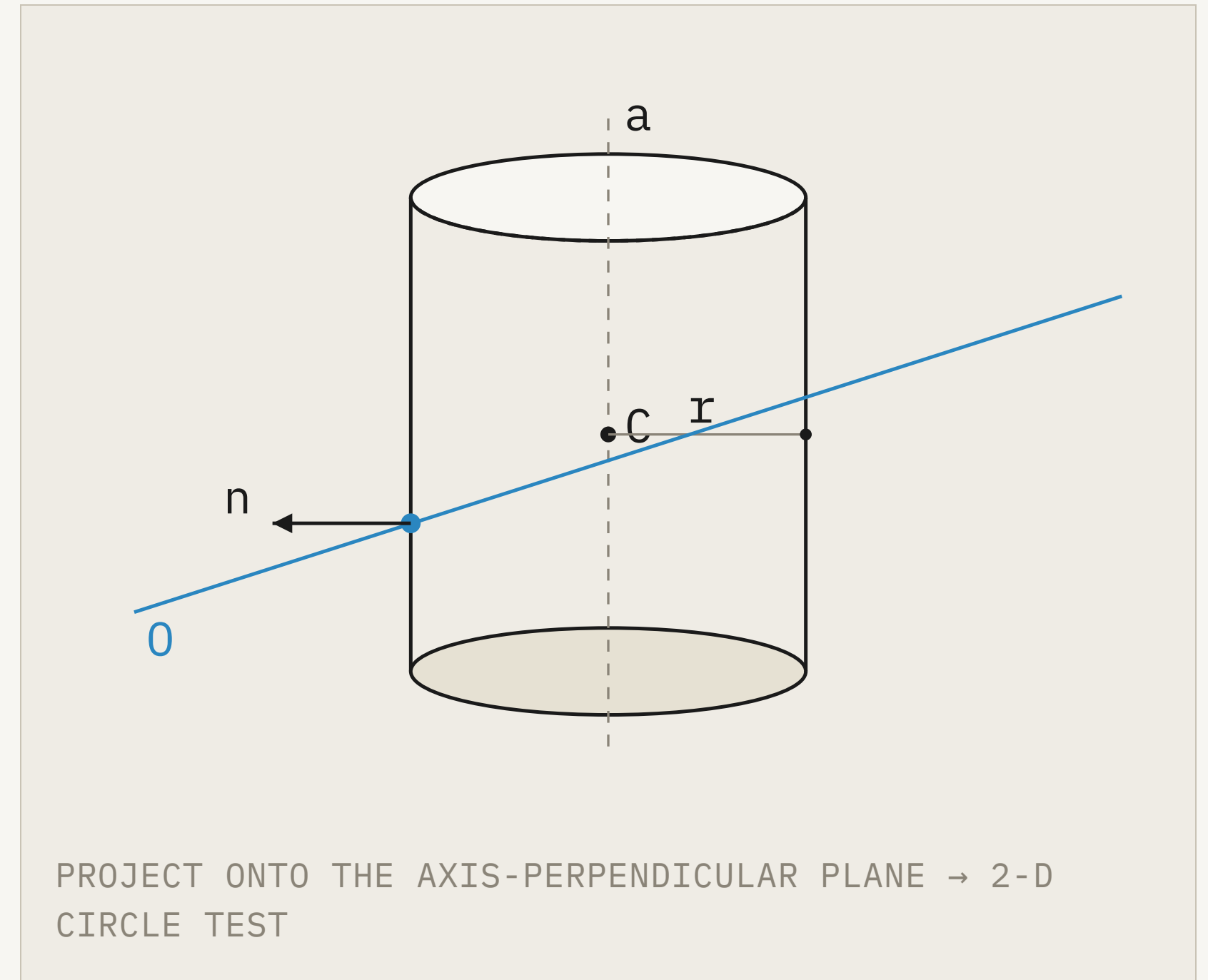
PRIMITIVE 3 / 9 • PROJECTED QUADRATIC

Locus of points at distance r from a line of axis $\mathbf{a}$.
We project the ray onto the plane perpendicular to the axis and solve a 2-D circle intersection.

$$\mathbf{d}' = \mathbf{d} - (\mathbf{d} \cdot \mathbf{a})\mathbf{a}, \quad \overrightarrow{OC}' = \overrightarrow{OC} - (\overrightarrow{OC} \cdot \mathbf{a})\mathbf{a}$$

$$\|\mathbf{d}'\|^2 t^2 + 2(\mathbf{d}' \cdot \overrightarrow{OC}')t + \|\overrightarrow{OC}'\|^2 - r^2 = 0$$

- Caps (when bounded) are tested as two planes at $\mathbf{C} \pm \frac{h}{2}\mathbf{a}$.
- Lateral normal: $\mathbf{n} = \overbrace{(\mathbf{P} - \mathbf{C}) - ((\mathbf{P} - \mathbf{C}) \cdot \mathbf{a})\mathbf{a}}$
- Cap normal: $\mathbf{n} = \pm \mathbf{a}$



11 Cone

Set of points whose vector from the apex C makes a fixed angle θ with the axis a . Squaring the cosine yields a quadratic in t .

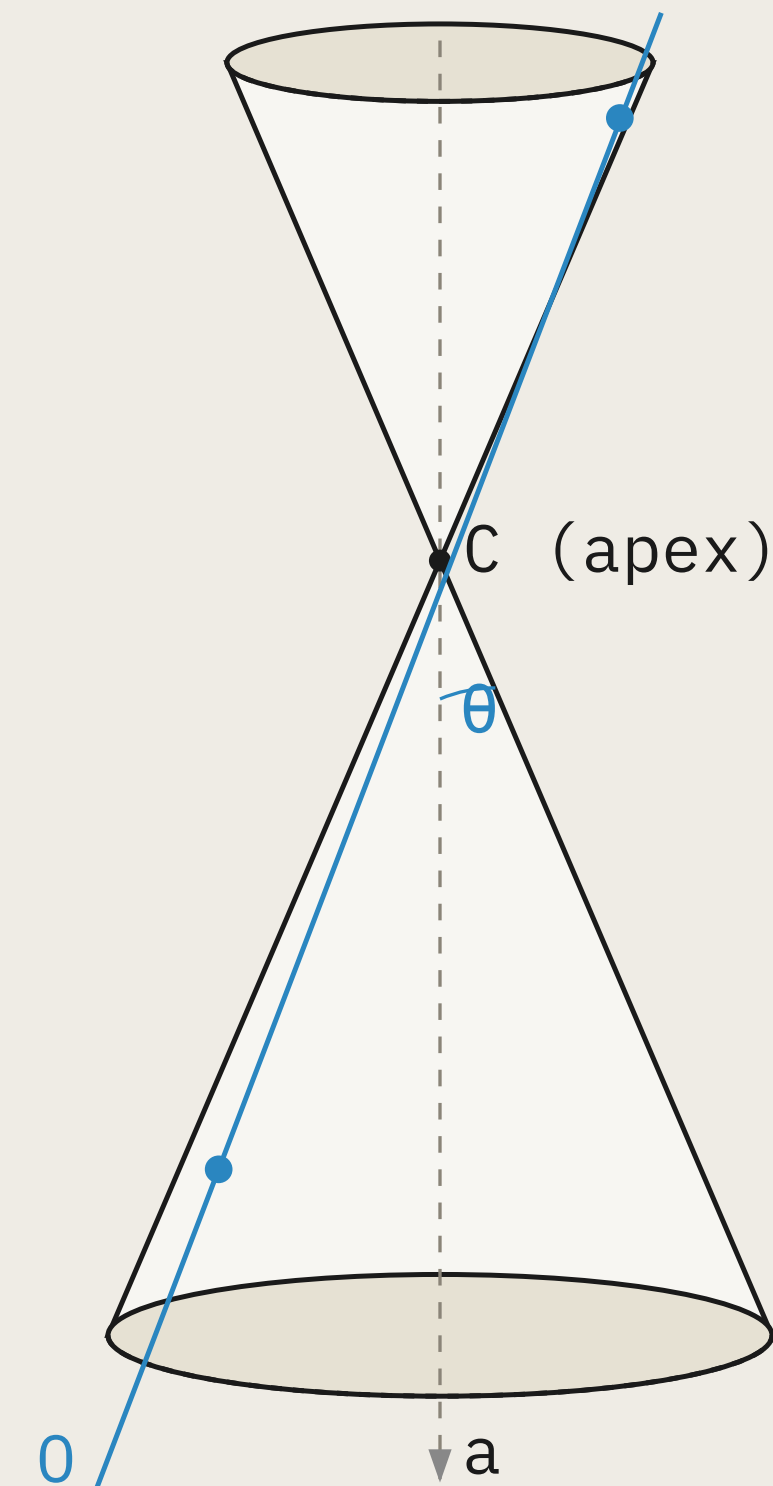
$$((P - C) \cdot \mathbf{a})^2 = k \|P - C\|^2, \quad k = \cos^2 \theta$$

$$At^2 + 2Bt + C = 0$$

$$A = (\mathbf{d} \cdot \mathbf{a})^2 - k, \quad B = (\mathbf{d} \cdot \mathbf{a})(\overrightarrow{OC} \cdot \mathbf{a}) - k(\mathbf{d} \cdot \overrightarrow{OC})$$

$$C = (\overrightarrow{OC} \cdot \mathbf{a})^2 - k \|\overrightarrow{OC}\|^2$$

Normal (gradient of the implicit form): $\mathbf{n} = \widehat{h\mathbf{a} - k\overrightarrow{P - C}}, \quad h = (P - C) \cdot \mathbf{a}$



TWO ROOTS: FRONT AND BACK NAPPES OF THE DOUBLE CONE

In the local XZ-plane frame, the torus of major radius R and minor radius r is an algebraic surface of degree 4.

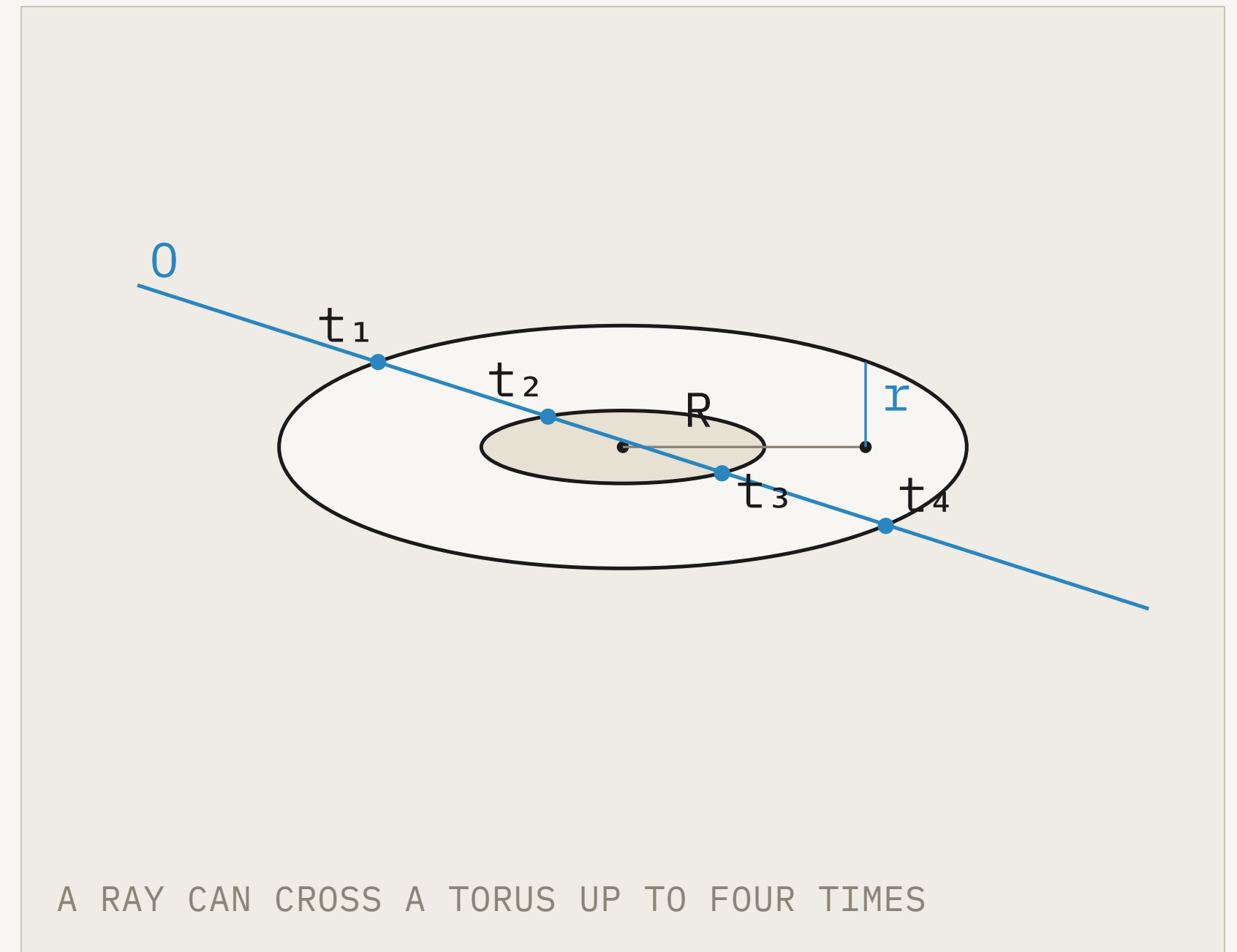
$$(x^2 + y^2 + z^2 + R^2 - r^2)^2 = 4R^2(x^2 + z^2)$$

Substituting $P(t) = \mathbf{O} + t\mathbf{d}$:

$$At^4 + Bt^3 + Ct^2 + Dt + E = 0$$

- Solved analytically via Ferrari's method: depress with $t = s - \frac{B}{4A}$, solve the resolvent cubic, recover four roots.
- Take the smallest real root with $t > \varepsilon$.

Normal (gradient of F): $\mathbf{n} = \overline{4P(\|P\|^2 + R^2 - r^2) - 8R^2(P_x, 0, P_z)}$



13 Triangle

Möller–Trumbore solves the intersection directly in barycentric coordinates (u, v) , never building the triangle's plane explicitly.

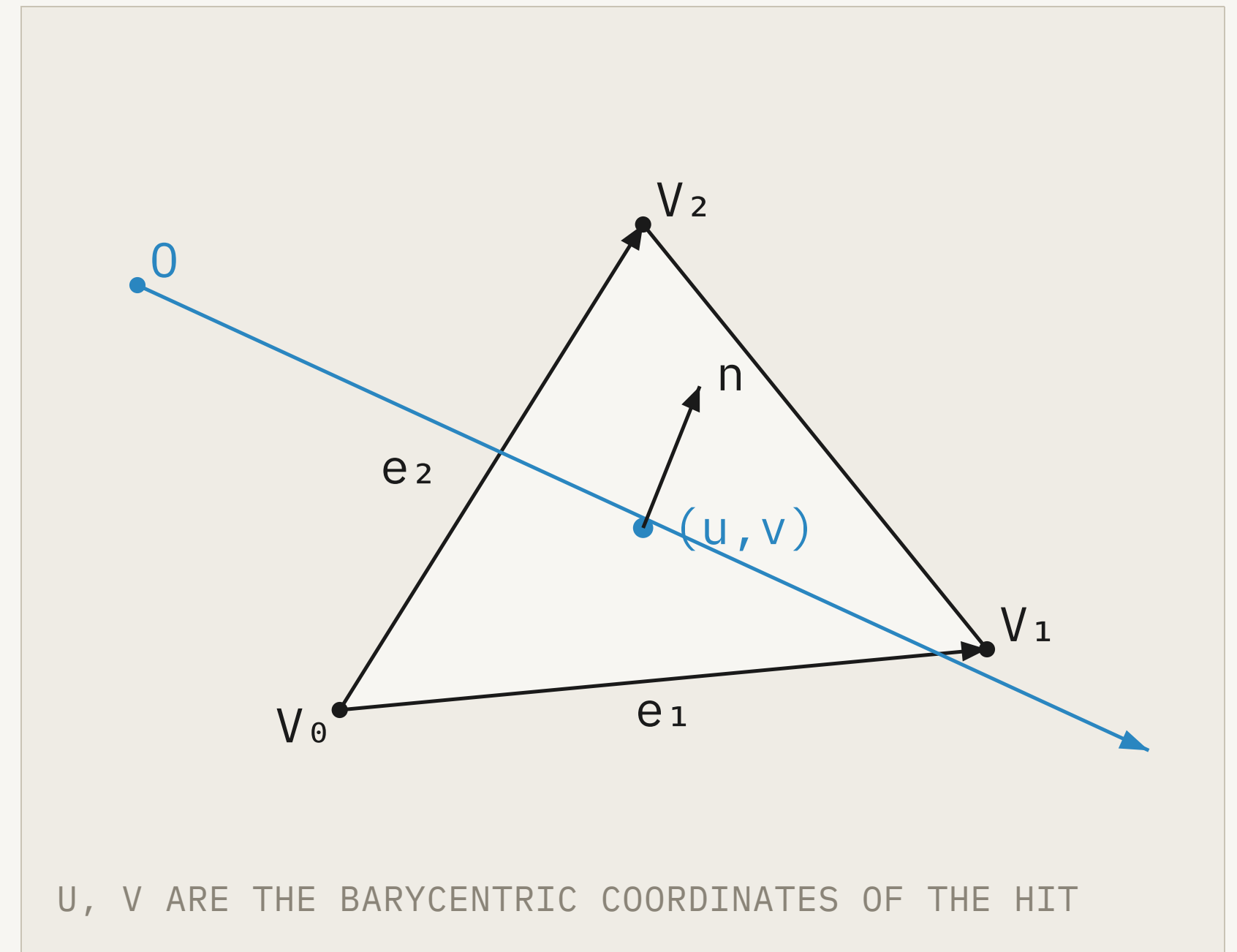
$$\mathbf{e}_1 = V_1 - V_0, \quad \mathbf{e}_2 = V_2 - V_0, \quad \mathbf{p} = \mathbf{d} \times \mathbf{e}_2$$

$$\det = \mathbf{e}_1 \cdot \mathbf{p}, \quad \mathbf{T} = \mathbf{O} - V_0, \quad \mathbf{q} = \mathbf{T} \times \mathbf{e}_1$$

$$u = \frac{\mathbf{T}\mathbf{p}}{\det}, \quad v = \frac{\mathbf{d}\mathbf{q}}{\det}, \quad t = \frac{\mathbf{e}_2\mathbf{q}}{\det}$$

Valid hit when: $u \geq 0, v \geq 0, u + v \leq 1, t > \varepsilon$

Smooth normal (OBJ meshes): $\mathbf{n} = u\mathbf{n}_1 + v\mathbf{n}_2 + (1 - u - v)\mathbf{n}_0$



14 Tanglecube

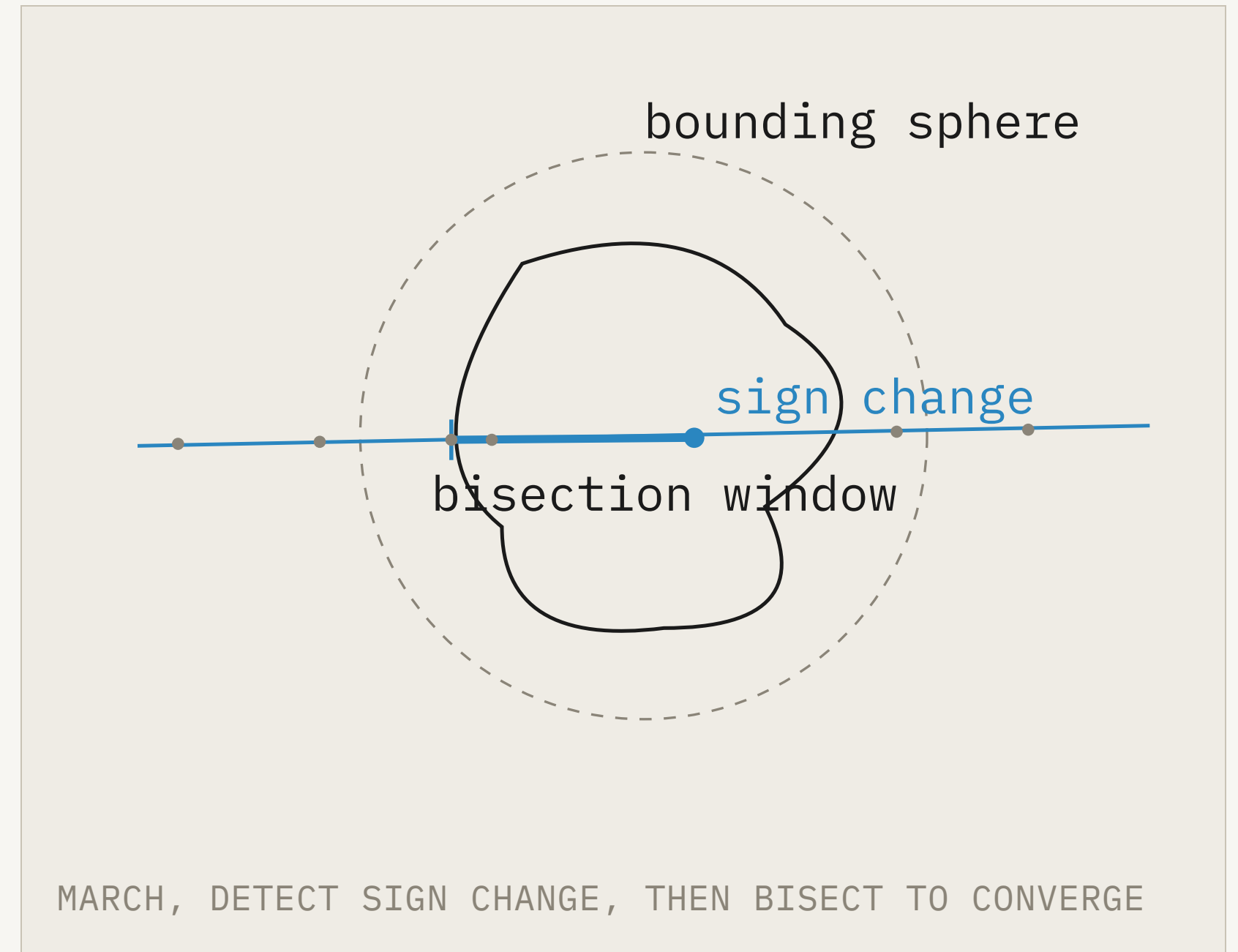
An algebraic surface of degree 4 with no closed-form ray intersection. We march the ray in discrete steps and refine by bisection on sign changes of F .

$$F(P) = x^4 - 5x^2 + y^4 - 5y^2 + z^4 - 5z^2 + 11.8 = 0$$

```
// 1. coarse march along the ray
for step in 0..255:
    t += stepSize
    if sign(F(P(t))) flips:
        bracketed → break

// 2. 64 bisection iterations
while i < 64:
    m = (t_a + t_b)/2
    if sign(F(m)) = sign(F(t_a)): t_a = m
    else: t_b = m
```

Normal: $\nabla F = (4x^3 - 10x, 4y^3 - 10y, 4z^3 - 10z)$



15 Mandelbulb

3-D analogue of the Mandelbrot set. The power is computed in spherical coordinates; a distance estimator (DE) lets us march safely.

$$z_0 = c = P, \quad z_{n+1} = z_n^k + c$$

$$r = \|z\|, \quad \theta = \arccos\left(\frac{z_y}{r}\right), \quad \varphi = \text{atan2}(z_z, z_x)$$

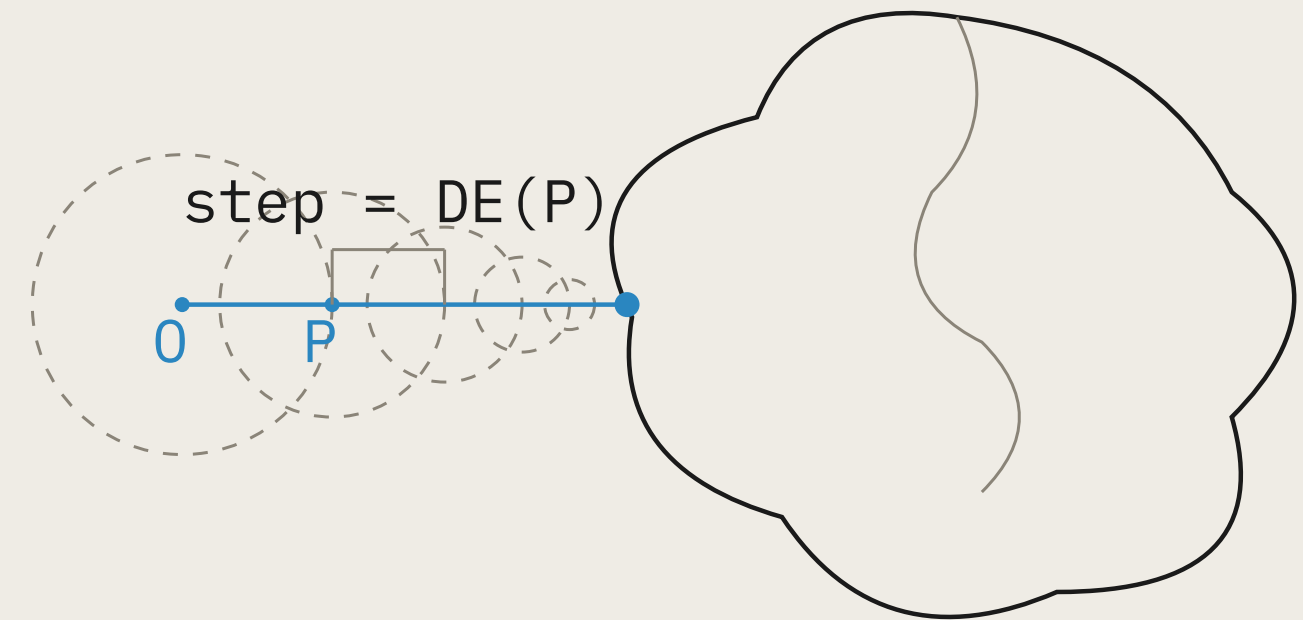
$$z^k = r^k (\sin k\theta \cos k\varphi, \cos k\theta, \sin k\theta \sin k\varphi)$$

$$dr \leftarrow k r^{k-1} dr + 1$$

$$\text{DE}(P) \approx \frac{1}{2} \log(r) \frac{r}{dr}$$

Sphere tracing: $t += \text{DE}(P)$ until $\text{DE} < 10^{-5}$ (hit) or $t > t_{\max}$ (miss).

Normal: finite differences of DE with $\varepsilon = 10^{-3}$.



EACH STEP IS THE SAFE RADIUS GIVEN BY THE DE

16 Möbius strip

A parametric, non-orientable surface. Intersection is a 3×3 nonlinear system solved by Newton iteration from eight seeds around the loop.

$$M(u, v) = \left((R + v \cos \frac{u}{2}) \cos u, (R + v \cos \frac{u}{2}) \sin u, v \sin \frac{u}{2} \right)$$

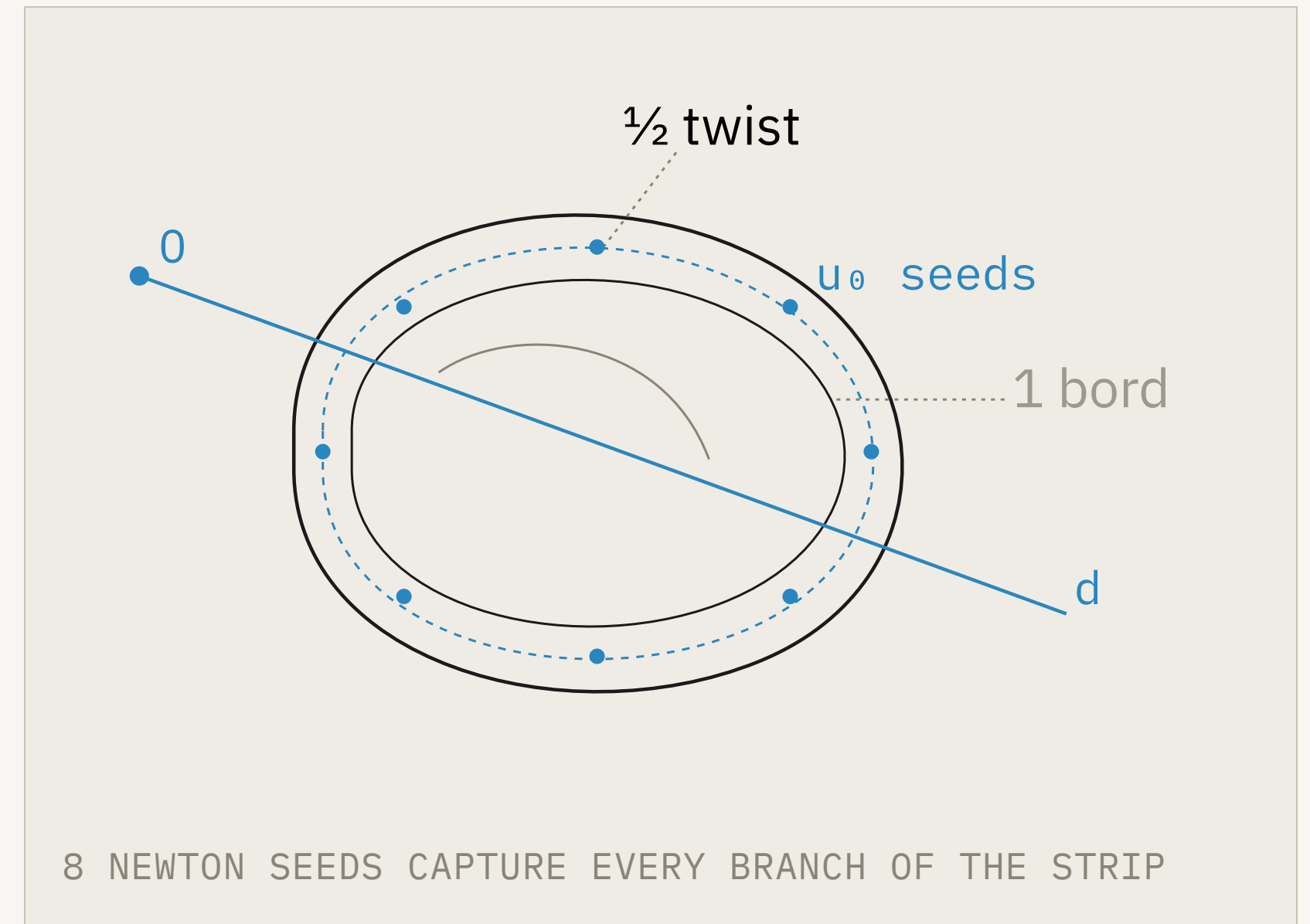
$$u \in [0, 2\pi), \quad v \in [-w, w]$$

$$\mathbf{F}(t, u, v) = M(u, v) - \mathbf{O} - t \mathbf{d} = \mathbf{0}$$

$$\begin{bmatrix} \Delta t \\ \Delta u \\ \Delta v \end{bmatrix} = -J_{\mathbf{F}}^{-1} \mathbf{F}$$

8 starting angles $u_0 \in \{0, \frac{\pi}{4}, \dots, \frac{7\pi}{4}\} \times 15$ Newton steps. Keep valid hits with $t > \varepsilon$ and $|v| \leq w$.

Normal: $\mathbf{n} = \widehat{\partial_u M \times \partial_v M}$



Transformations. How geometry is moved.

18 The decorator pattern

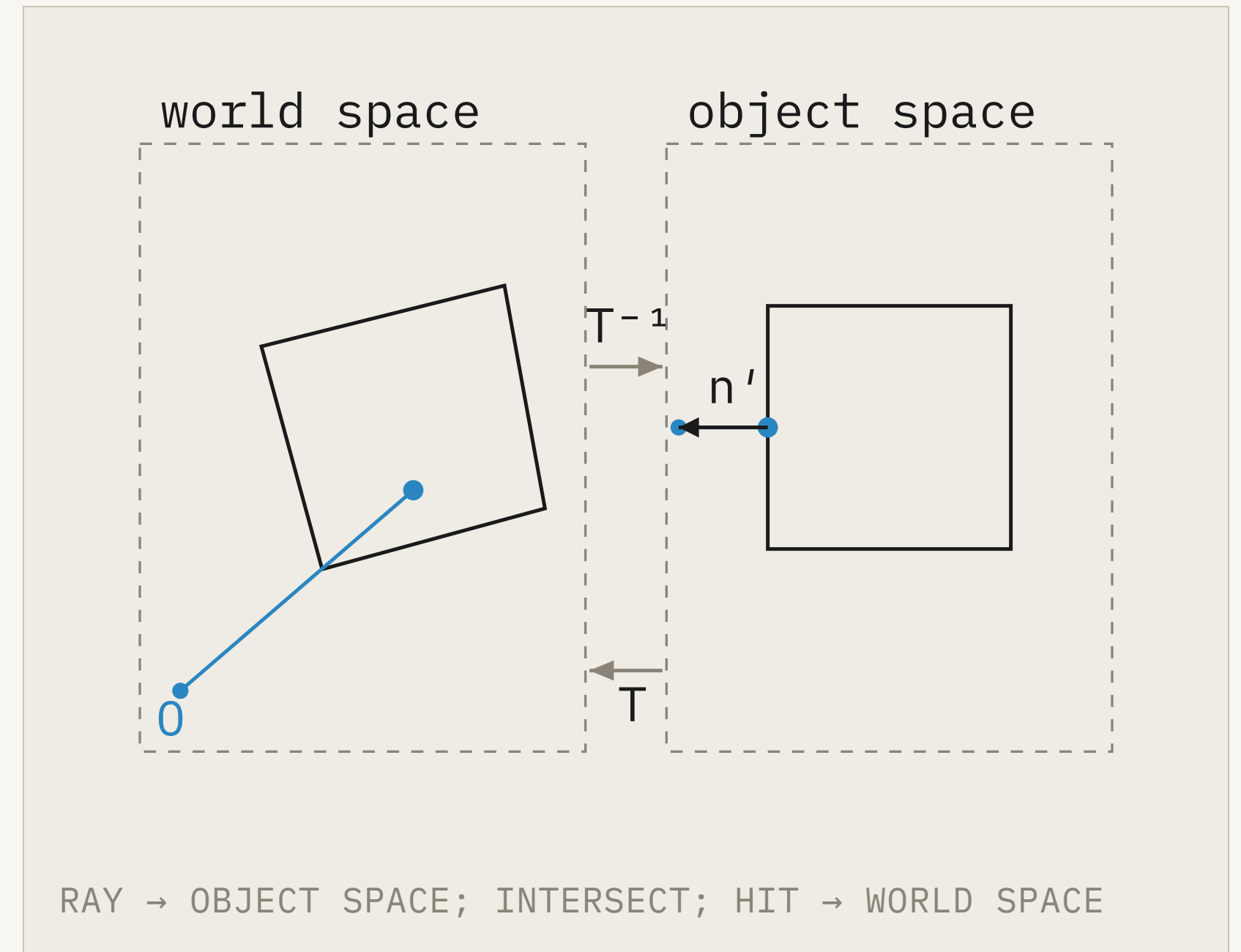
PRINCIPLE · INVERSE ON THE RAY

Rather than modifying geometry, each transform wraps any `IPrimitive`. The inverse transform is applied to the incoming ray, intersection is computed in object space, and the resulting hit is mapped back to world space.

```
function intersect(ray):  
    localRay = T-1(ray)    // ray into object space  
    hit = inner.intersect(localRay)  
    hit.point = T(hit.point) // point back to world  
    hit.normal = normalTransform(hit.normal)  
    return hit
```

Inverse-transpose rule. If points transform as $p' = Mp$, the normal must use $(M^{-1})^T$ to stay orthogonal to tangent vectors:

$$\mathbf{n}' = \widehat{(M^{-1})^T \mathbf{n}}$$



19 Translation

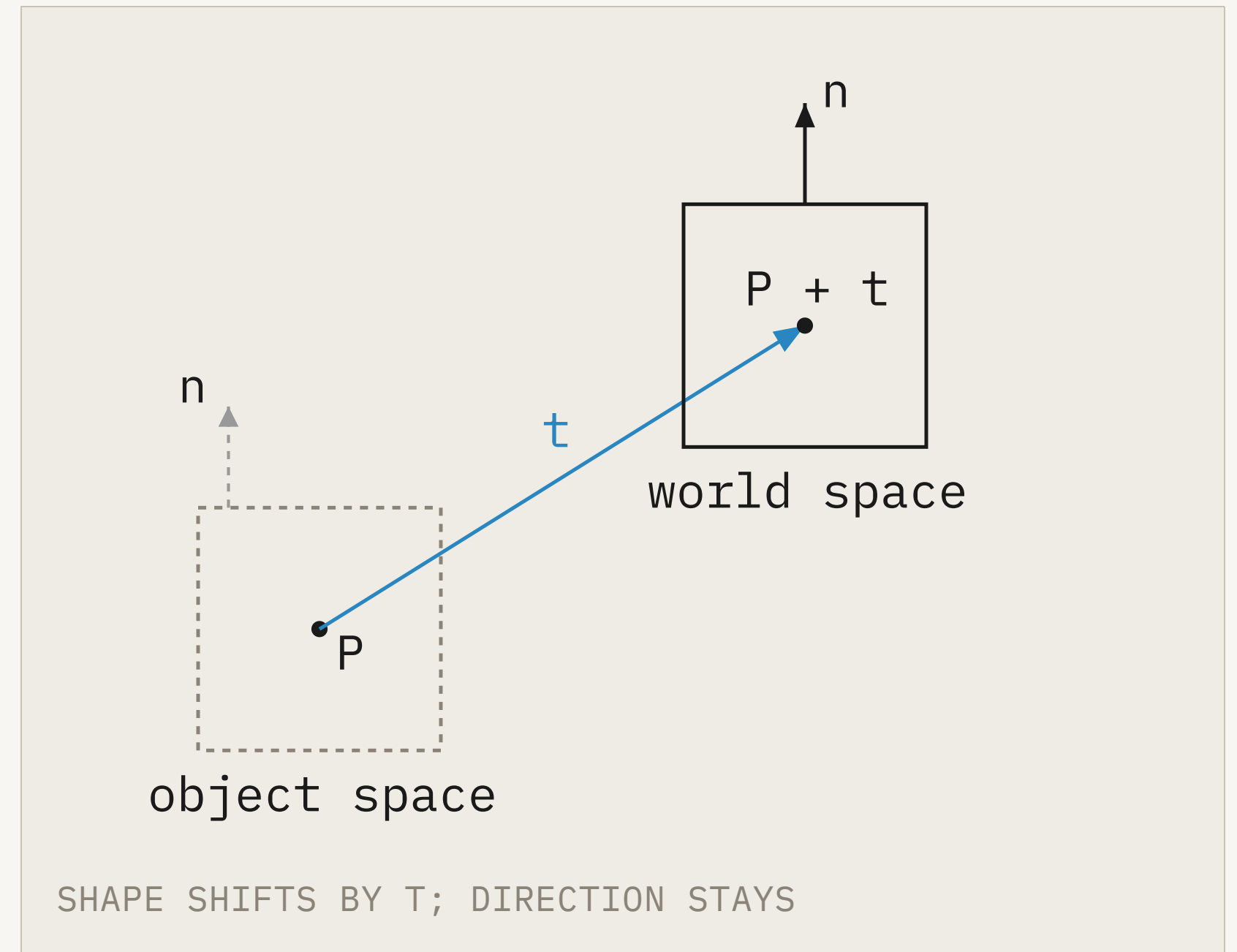
An offset vector $\mathbf{t} = (t_x, t_y, t_z)$ shifts every point:
 $P' = P + \mathbf{t}$. Directions — and therefore normals — are unaffected.

$$\mathbf{O}' = \mathbf{O} - \mathbf{t}, \quad \mathbf{d}' = \mathbf{d}$$

$$P_{\text{world}} = P'_{\text{local}} + \mathbf{t}$$

$$\mathbf{n}_{\text{world}} = \mathbf{n}'_{\text{local}}$$

- Only the ray's origin is shifted; the direction is invariant.
- Orientation untouched — normals pass through.



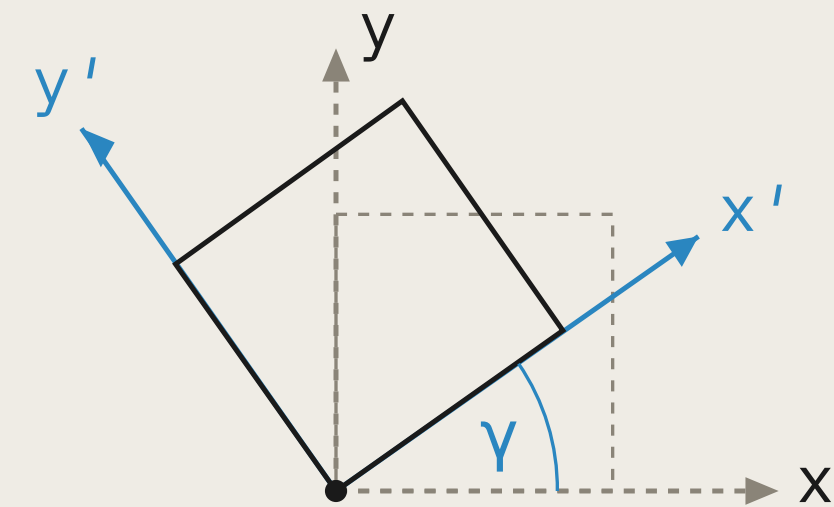
Three Euler angles (α, β, γ) build a composite matrix $R = R_x(\alpha) R_y(\beta) R_z(\gamma)$. Because R is orthogonal, $R^{-1} = R^T$.

$$R_x = \begin{pmatrix} 1 & 0 & 0 \\ 0 & \cos \alpha & -\sin \alpha \\ 0 & \sin \alpha & \cos \alpha \end{pmatrix}$$

$$\mathbf{O}' = R^T \mathbf{O}, \quad \mathbf{d}' = R^T \mathbf{d}$$

$$P_{\text{world}} = R P'_{\text{local}}, \quad \mathbf{n}_{\text{world}} = \widehat{R \mathbf{n}'_{\text{local}}}$$

Normals use R — not R^T — because $(R^{-1})^T = (R^T)^T = R$.



$$R = R_x(\alpha) \cdot R_y(\beta) \cdot R_z(\gamma)$$

ORTHOGONAL: LENGTHS AND ANGLES PRESERVED

A diagonal matrix with per-axis factors (s_x, s_y, s_z) , all non-zero. Ray components divide by s_i ; the normal does too — for a diagonal matrix, $(S^{-1})^T = S^{-1}$.

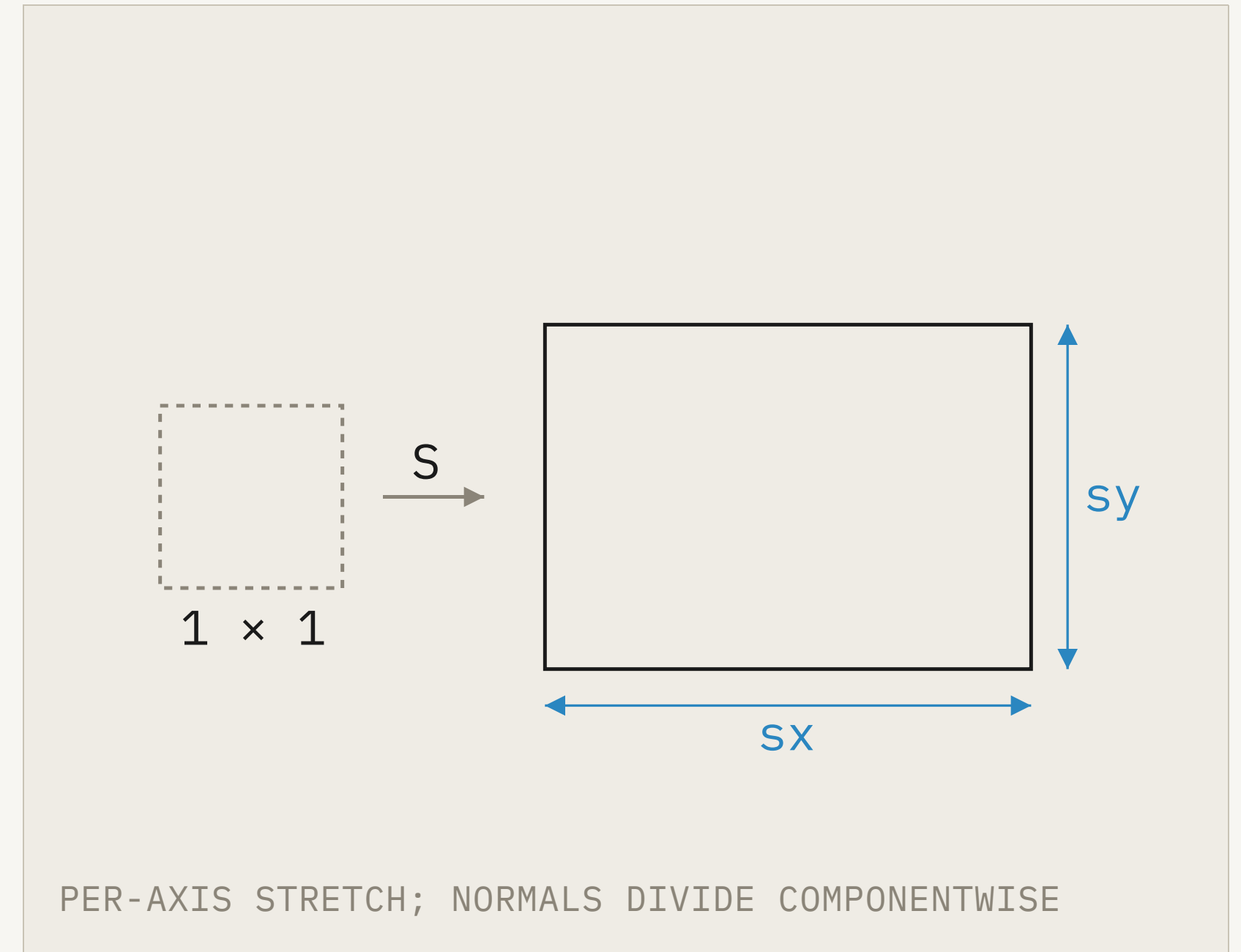
$$S = \text{diag}(s_x, s_y, s_z)$$

$$\mathbf{o}' = \left(\frac{o_x}{s_x}, \frac{o_y}{s_y}, \frac{o_z}{s_z} \right), \quad \mathbf{d}' = \left(\frac{d_x}{s_x}, \frac{d_y}{s_y}, \frac{d_z}{s_z} \right)$$

$$P_{\text{world}} = (s_x P'_x, s_y P'_y, s_z P'_z)$$

$$\mathbf{n}_{\text{world}} = \left(\frac{n'_x}{s_x}, \frac{n'_y}{s_y}, \frac{n'_z}{s_z} \right)$$

Tangent and bitangent transform like the normal, not like points.



22 Shear

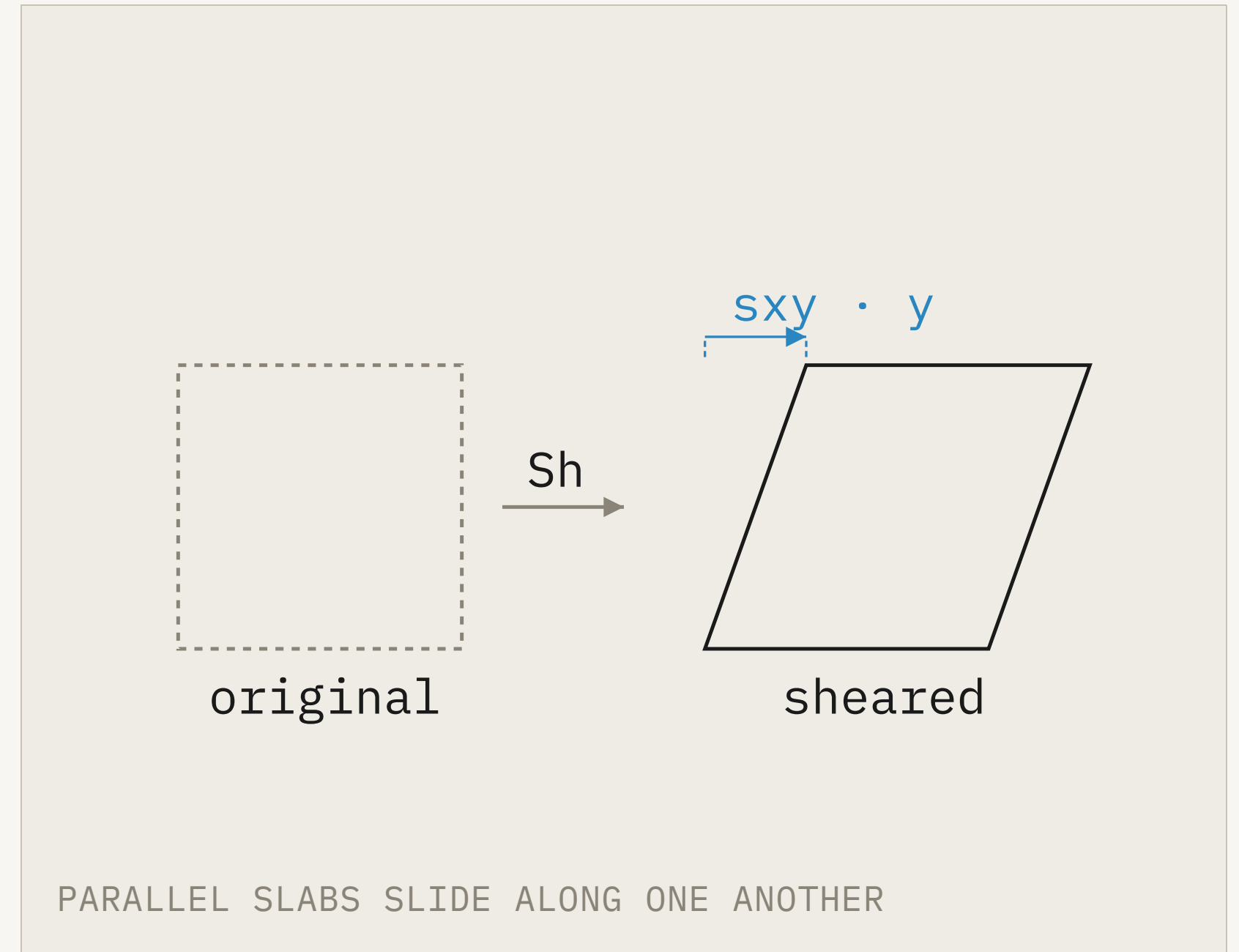
Six factors s_{ij} each displace axis i in proportion to axis j . The inverse S_h^{-1} and its transpose $(S_h^{-1})^T$ are precomputed at construction.

$$S_h = \begin{pmatrix} 1 & s_{xy} & s_{xz} \\ s_{yx} & 1 & s_{yz} \\ s_{zx} & s_{zy} & 1 \end{pmatrix}$$

$$\mathbf{o}' = S_h^{-1} \mathbf{o}, \quad \mathbf{d}' = S_h^{-1} \mathbf{d}$$

$$P_{\text{world}} = S_h P'_{\text{local}}$$

$$\mathbf{n}_{\text{world}} = \widehat{(S_h^{-1})^T \mathbf{n}'_{\text{local}}}$$



23 Arbitrary 4×4 matrix

DECORATOR 5 / 5 • HOMOGENEOUS • MOST GENERAL

Any affine transform — or any combination — fits into a single 4×4 matrix. The inverse M^{-1} is computed once via Gaussian elimination with partial pivoting.

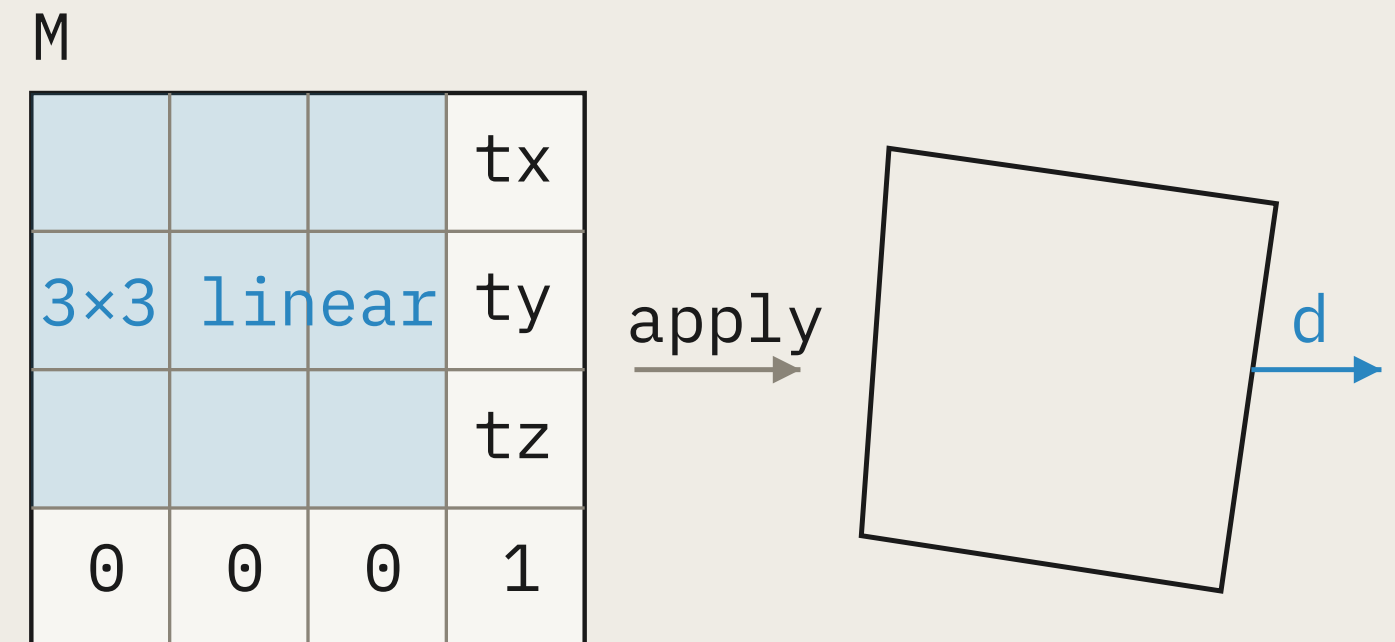
point ($w = 1$): $\text{tPoint}(M, p)_i = M_{i0}p_x + M_{i1}p_y + M_{i2}p_z + M_{i3}$

direction ($w = 0$): $\text{tDir}(M, d)_i = M_{i0}d_x + M_{i1}d_y + M_{i2}d_z$

$$\mathbf{O}' = \text{tPoint}(M^{-1}, \mathbf{O}), \quad \mathbf{d}' = \text{tDir}(M^{-1}, \mathbf{d})$$

$$\mathbf{n}_{\text{world}} = \overbrace{(M_{3 \times 3}^{-1})^T} \mathbf{n}'_{\text{local}}$$

The normal uses only the upper-left 3×3 block of M^{-1} — homogeneous translation has no effect on directions.



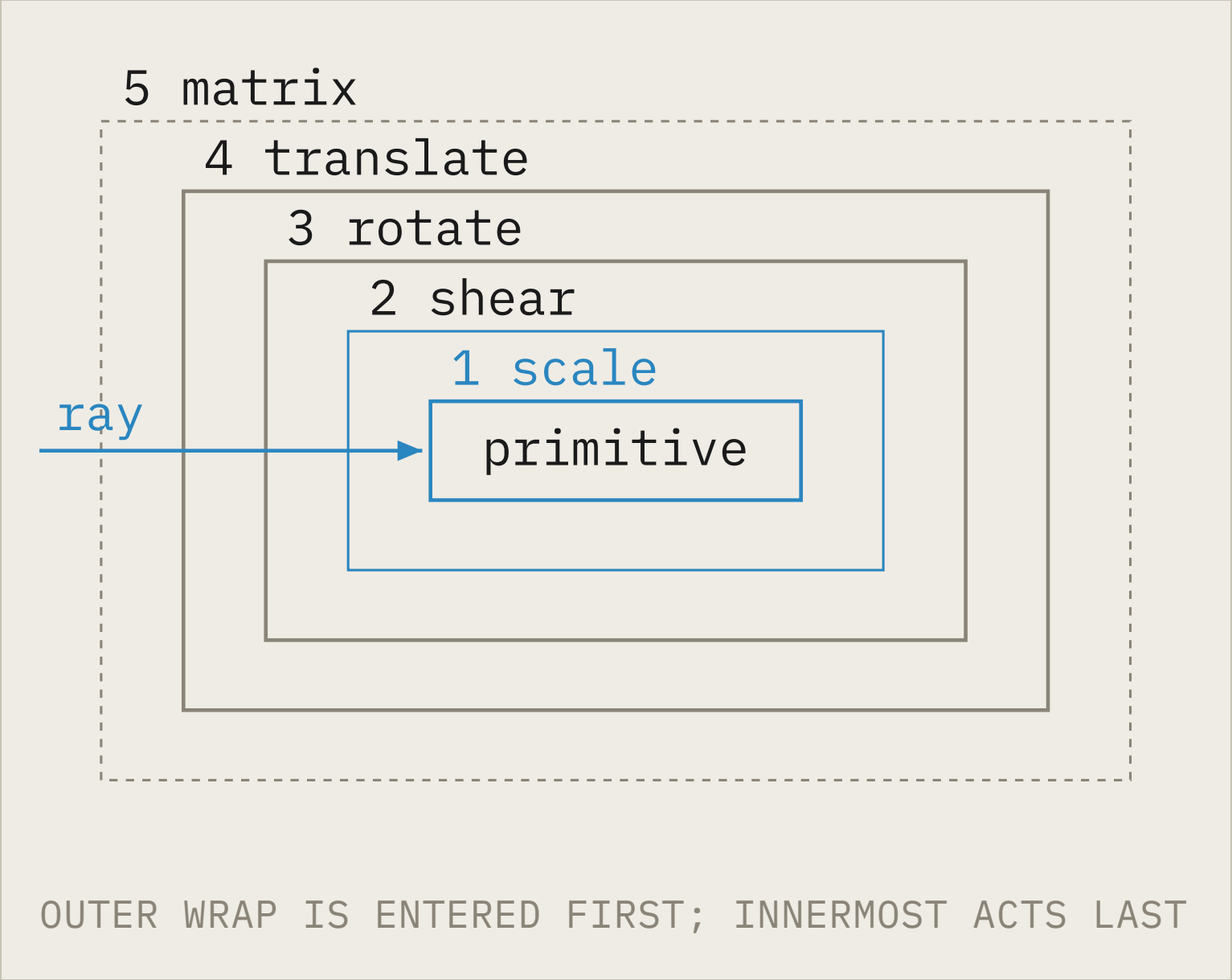
ONE MATRIX COLLAPSES A WHOLE TRANSFORM STACK

24 Application order

TransformBuilder::applyTransforms wraps the primitive in this fixed order — only the decorators present in the scene file are actually applied.

STEP	DECORATOR	OBJECT → WORLD
1	ScaleDecorator	scale first
2	ShearDecorator	then shear
3	RotationDecorator	then rotate
4	TranslationDecorator	then translate
5	TransformMatrixDecorator	arbitrary matrix last

The ray travels in reverse: the outermost decorator transforms it first — M^{-1} , then T^{-1} , then R^T , then S_h^{-1} , then S^{-1} — before reaching the primitive.



Lights. How the scene is illuminated.

26 The shading sum

DIRECT + AO + INDIRECT

Every surface point combines diffuse and specular contributions from each light, an ambient-occlusion term, and one recursive indirect bounce.

$$C_{\text{final}} = \rho \left(\sum_{\ell} C_{\text{diff}}^{(\ell)} + C_{\text{AO}} \right) + \sum_{\ell} C_{\text{spec}}^{(\ell)} + \rho C_{\text{indirect}}$$

ρ Material attenuation (albedo)

C_{diffuse} Lambert term: $(\mathbf{n} \cdot \mathbf{L}) C_{\ell}$

C_{specular} Phong term (only on glossy materials)

C_{AO} Hemisphere occlusion sample

C_{indirect} traceRay on the secondary ray

A light sample contains:

- `direction` — unit vector from hit point toward the light
- `color` — light colour pre-multiplied by intensity
- `distance` — distance to source (∞ for directional)
- `isAmbient` — distinguishes the AO term

27 Ambient occlusion

LIGHT 1 / 3 • MONTE-CARLO

Replaces classical ambient with a local Monte-Carlo estimate: sample N rays in the upper hemisphere and count how many escape unblocked.

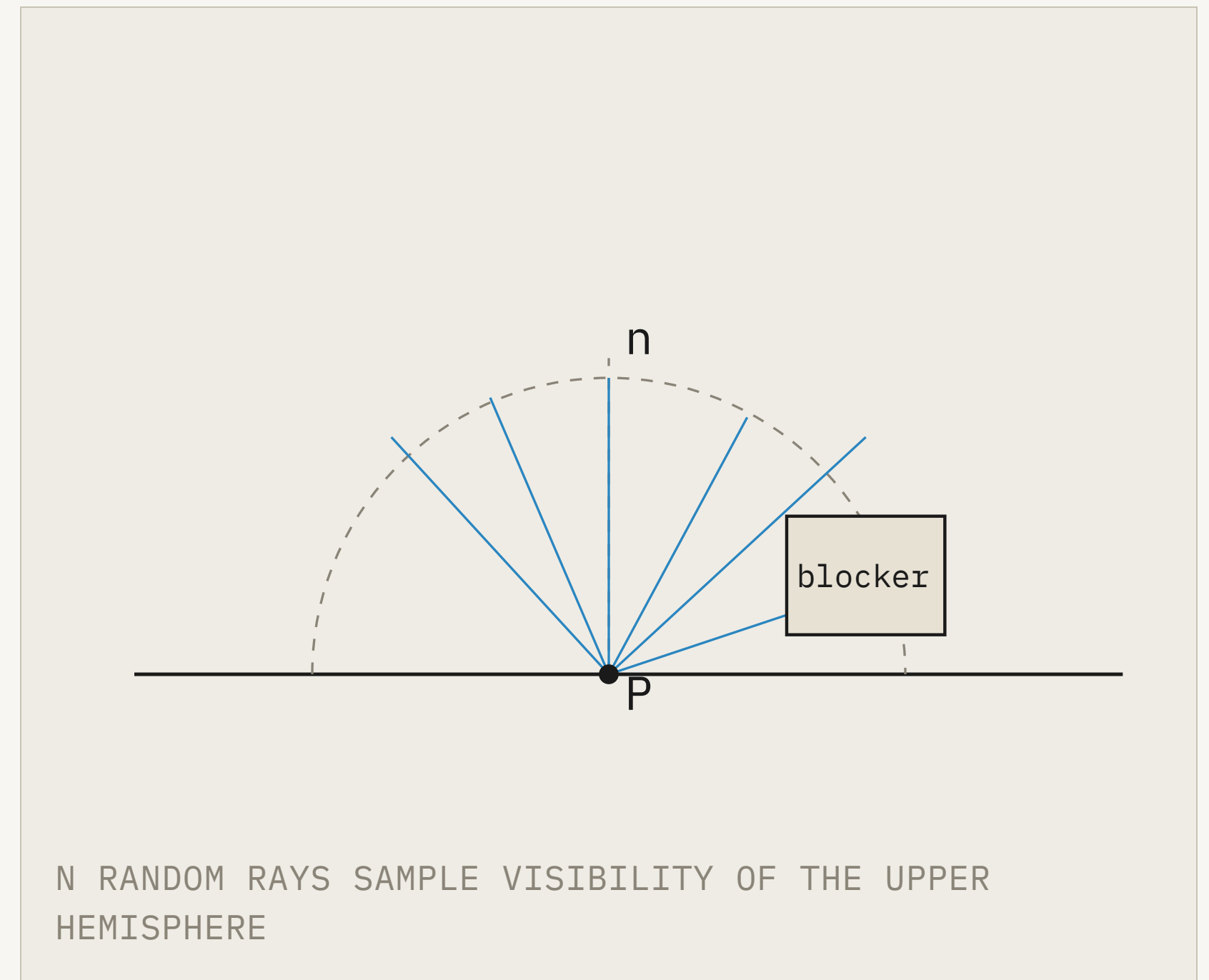
$$AO(P) = \frac{1}{N} \sum_{i=1}^N 1[\text{ray}(P, \xi_i) \text{ is unoccluded}]$$

$$C_{AO} = \text{light.color} \cdot AO(P)$$

Hemisphere sampling. Draw $\mathbf{p}$ uniformly in the unit ball, then flip if $\mathbf{p} \cdot \mathbf{n} < 0$:

$$\xi = \widehat{\text{sign}(\mathbf{p} \cdot \mathbf{n}) \mathbf{p}}$$

- $AO = 1$: open sky $\rightarrow$ full ambient.
- $AO = 0$: enclosed crevice $\rightarrow$ no ambient.



28 Directional light

Models an infinitely distant source (e.g. the sun): all light rays are parallel and there is no distance attenuation.

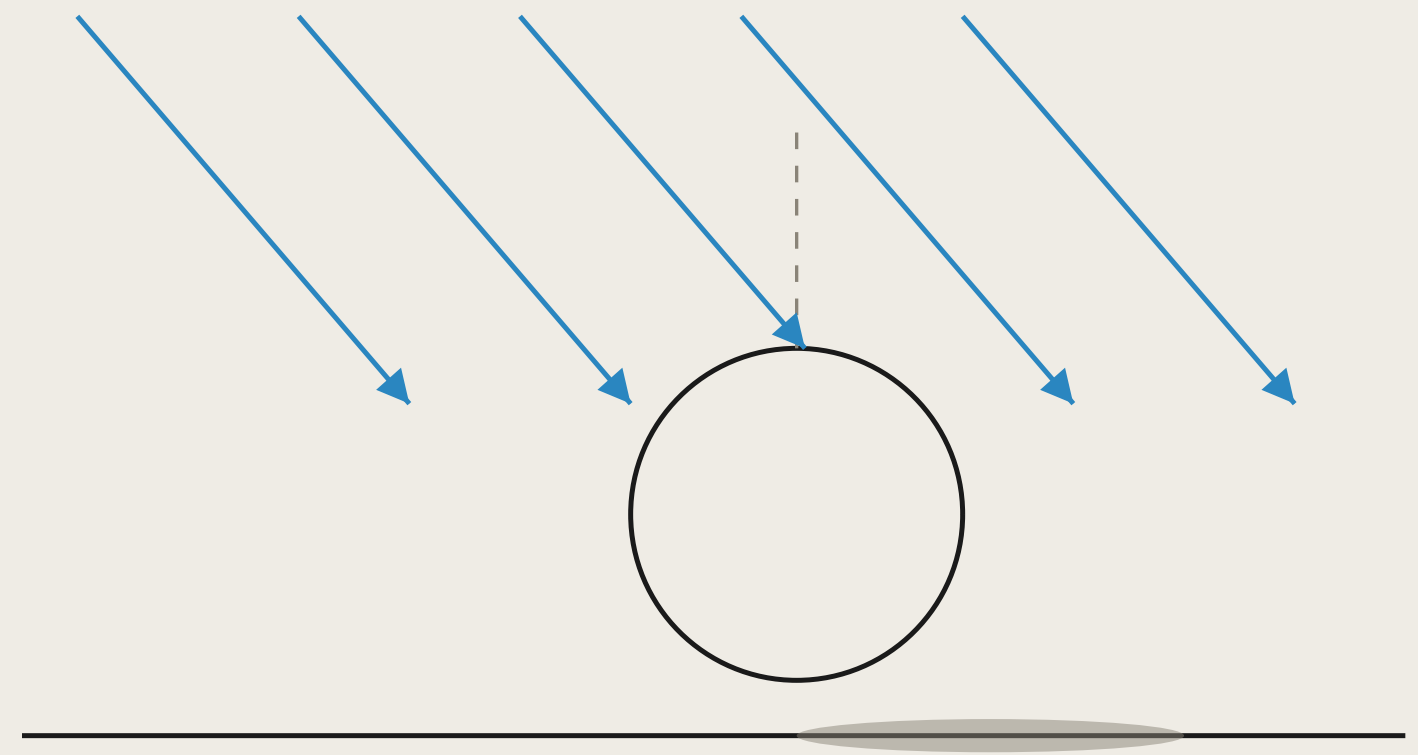
$$\text{sample} = \begin{cases} \mathbf{L} = -\mathbf{d}_\ell \\ \text{color} = C_\ell \cdot I \\ \text{distance} = \infty \end{cases}$$

$$C_{\text{diffuse}} = (\mathbf{n} \cdot \mathbf{L}) C_\ell \cdot I$$

Shadow ray. Cast from $P + \epsilon \mathbf{n}$ along $\mathbf{L}$ with infinite range — any opaque hit kills the light.

Transparent and refractive surfaces accumulate their colour into a `shadowFilter` instead of fully blocking the ray.

parallel rays · $\mathbf{L} = -\mathbf{d}_\ell$



NO FALLOFF; ONE DIRECTION FOR EVERY POINT IN THE SCENE

29 Point light

LIGHT 3 / 3 · INVERSE SQUARE

Radiates equally in every direction from a fixed position. Intensity decays with the squared distance to the receiver.

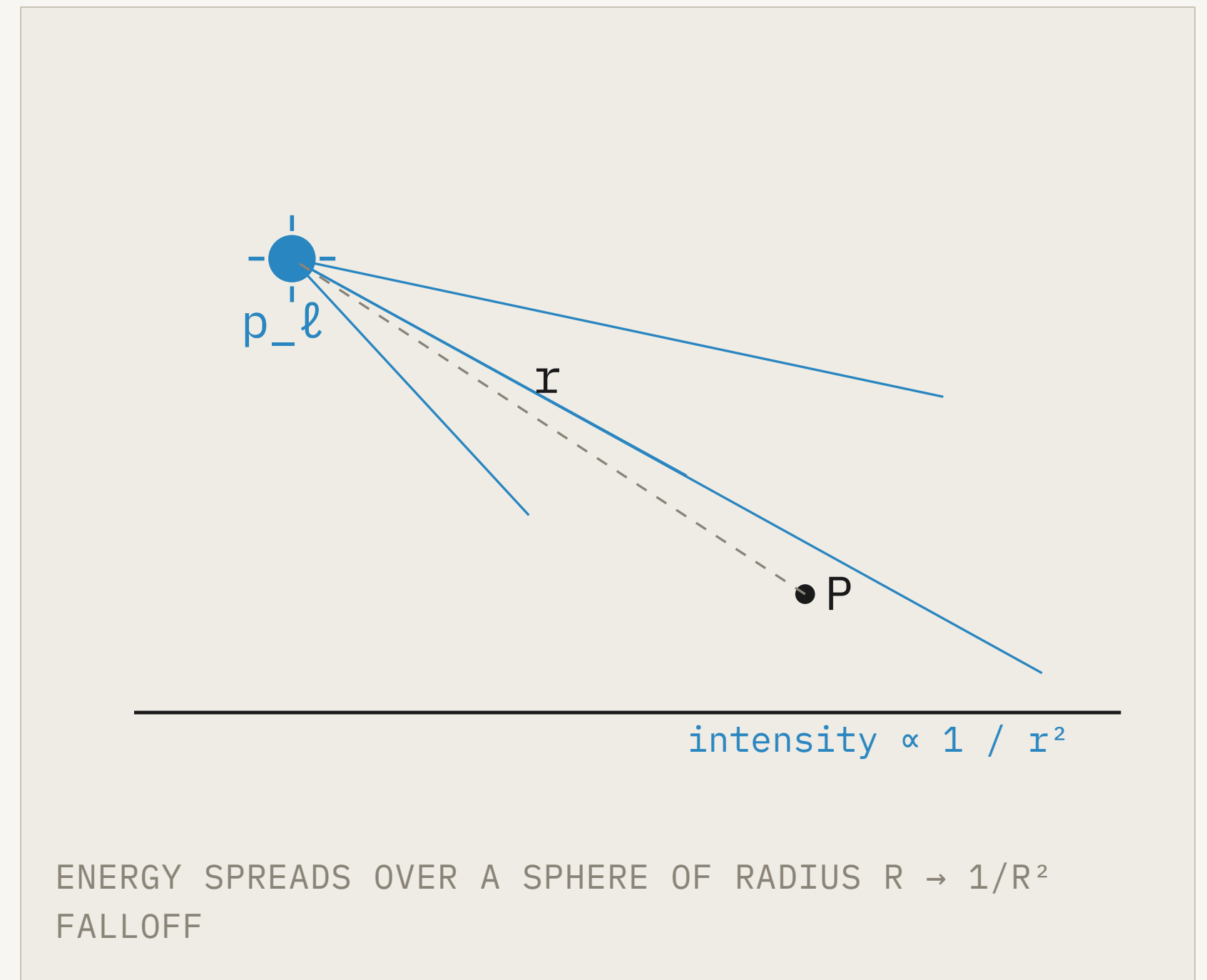
$$\mathbf{L} = \widehat{\mathbf{p}_\ell - P}, \quad r = \|\mathbf{p}_\ell - P\|$$

$$\text{color} = \frac{C_\ell \cdot I}{r^2}$$

$$C_{\text{diffuse}} = (\mathbf{n} \cdot \mathbf{L}) \frac{C_\ell \cdot I}{r^2}$$

Shadow ray ends at the light: an occluder is counted only if its hit is closer than r .

Transmissive blockers chain up to 8 bounces, multiplying their attenuation into a running shadowFilter.



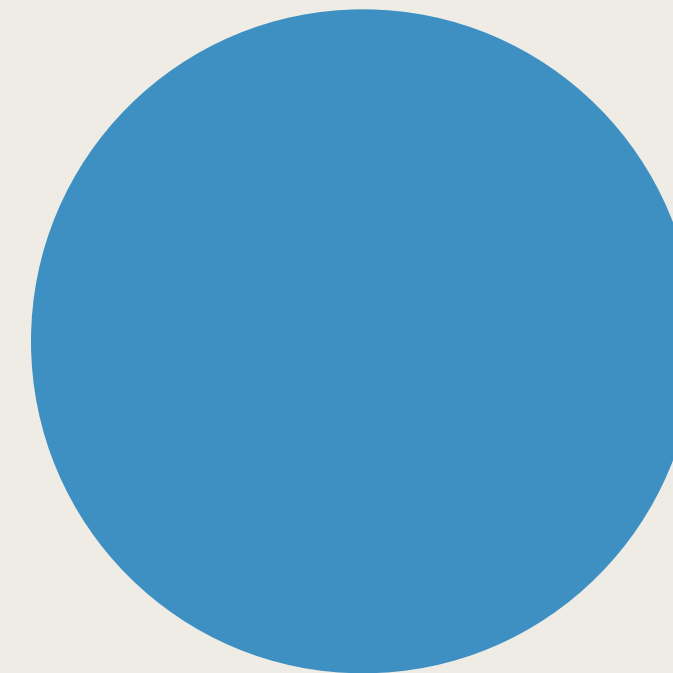
Materials. How surfaces scatter light.

The simplest material: returns a constant albedo, no specular highlight, no secondary ray. Used for unlit overlays and debugging.

$$\text{scatter}(r, h) = \begin{cases} \text{attenuation} = \mathbf{C}_0 \\ \text{secondaryRay} = \emptyset \end{cases}$$

$$C_{\text{shaded}} = \mathbf{C}_0 \sum_{\ell} (\mathbf{n} \cdot \mathbf{L}_{\ell}) C_{\ell}$$

The colour is still multiplied by the Lambert term in `traceRay`, so flat materials still respect shadows and light direction.



`attenuation = C0`

NO RECURSION — A SINGLE CONSTANT TINT

32 Phong material

Combines diffuse (Lambert) and a specular lobe centred on the mirror reflection of the light direction.

$$C_{\text{diffuse}} = (\mathbf{n} \cdot \mathbf{L}) C_{\ell} \rho$$

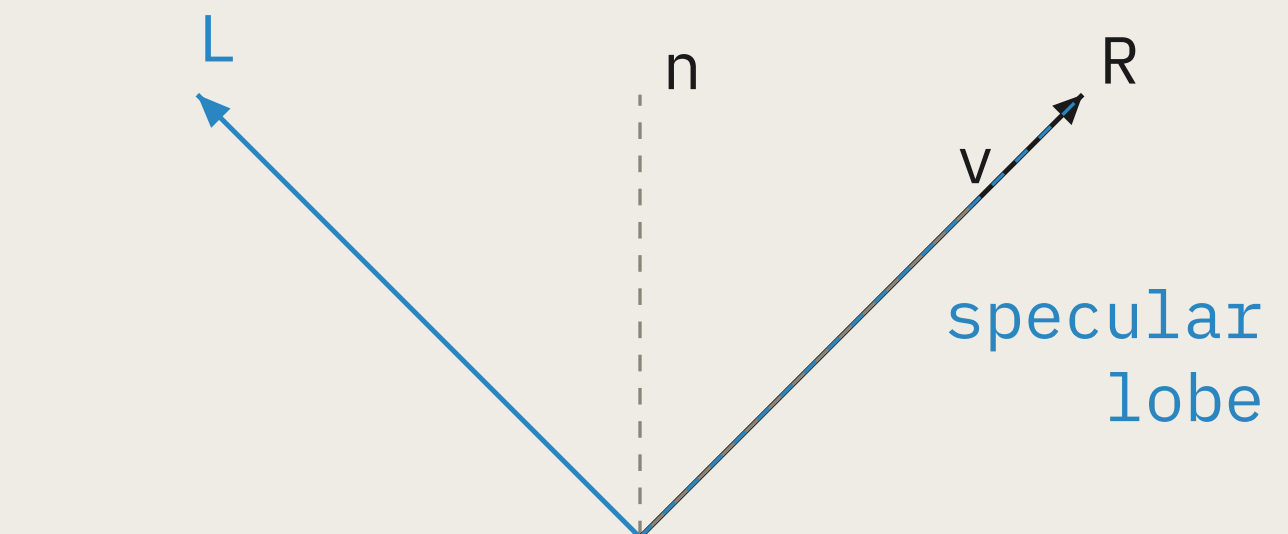
$$\mathbf{R} = -\mathbf{L} + 2(\mathbf{L} \cdot \mathbf{n}) \mathbf{n}$$

$$C_{\text{specular}} = k_s C_{\ell} \max(0, \mathbf{R} \cdot \mathbf{v})^s$$

k_s Specular coefficient (colour of the highlight)

s Shininess — higher $s \rightarrow$ narrower highlight

$\mathbf{v}$ View direction: $\mathbf{v} = -\hat{\mathbf{d}}$



THE HIGHLIGHT PEAKS WHEN $\mathbf{v}$ ALIGNS WITH $\mathbf{R}$

33 Reflection

Mirror surfaces emit a single secondary ray, obtained by reflecting the incoming direction about the surface normal.

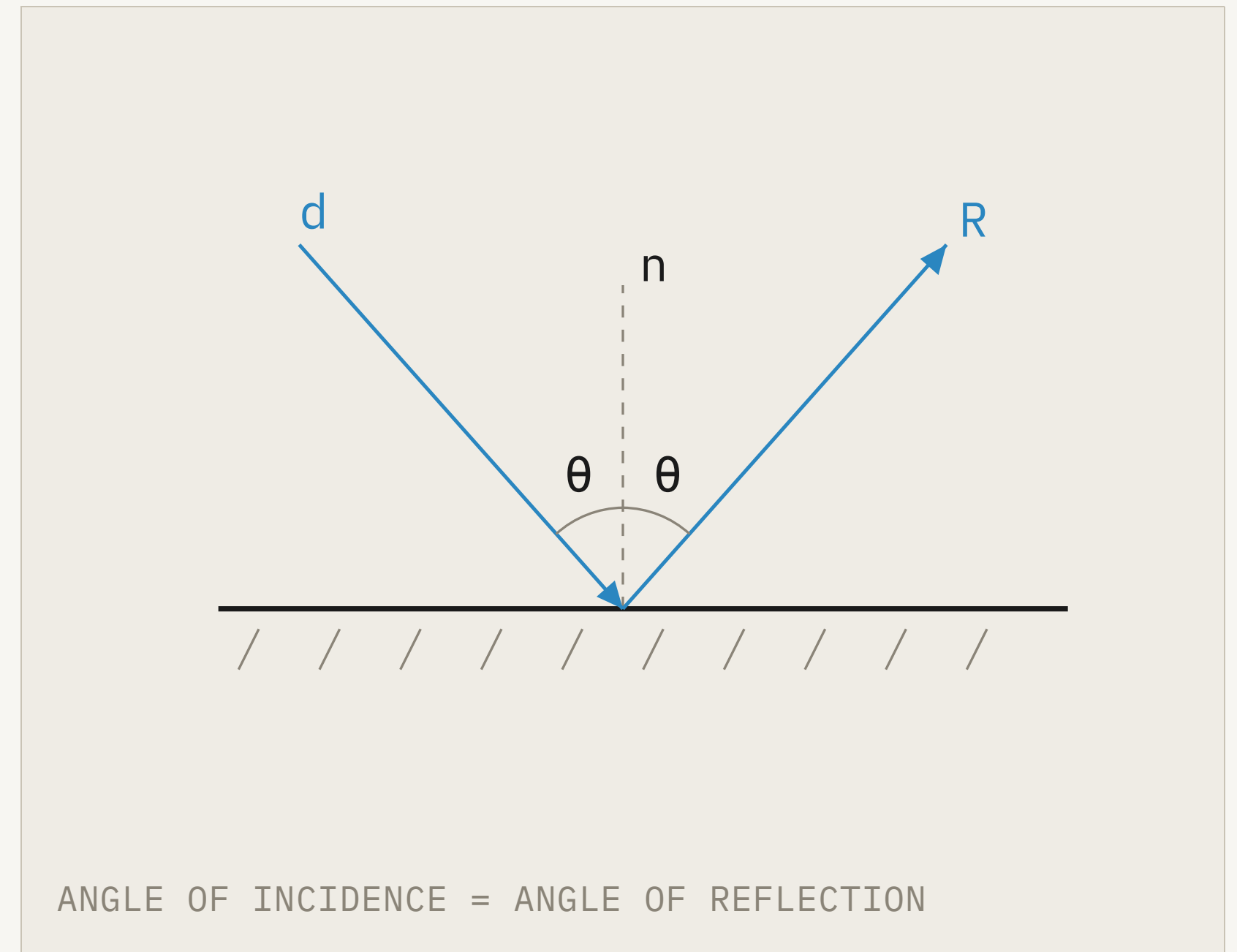
$$\mathbf{R} = \mathbf{d} - 2(\mathbf{d} \cdot \mathbf{n}) \mathbf{n}$$

$$\text{secondaryRay} = (P + \epsilon \mathbf{n}, \hat{\mathbf{R}})$$

$$C = \rho_{\text{mirror}} \cdot \text{traceRay}(\text{secondaryRay}, d - 1)$$

Recursion stops when depth = 0, capping the mirror corridors at MAX_DEPTH = 10 bounces.

The origin is offset by $\epsilon \mathbf{n}$ (the RayBias) to avoid re-hitting the same surface.



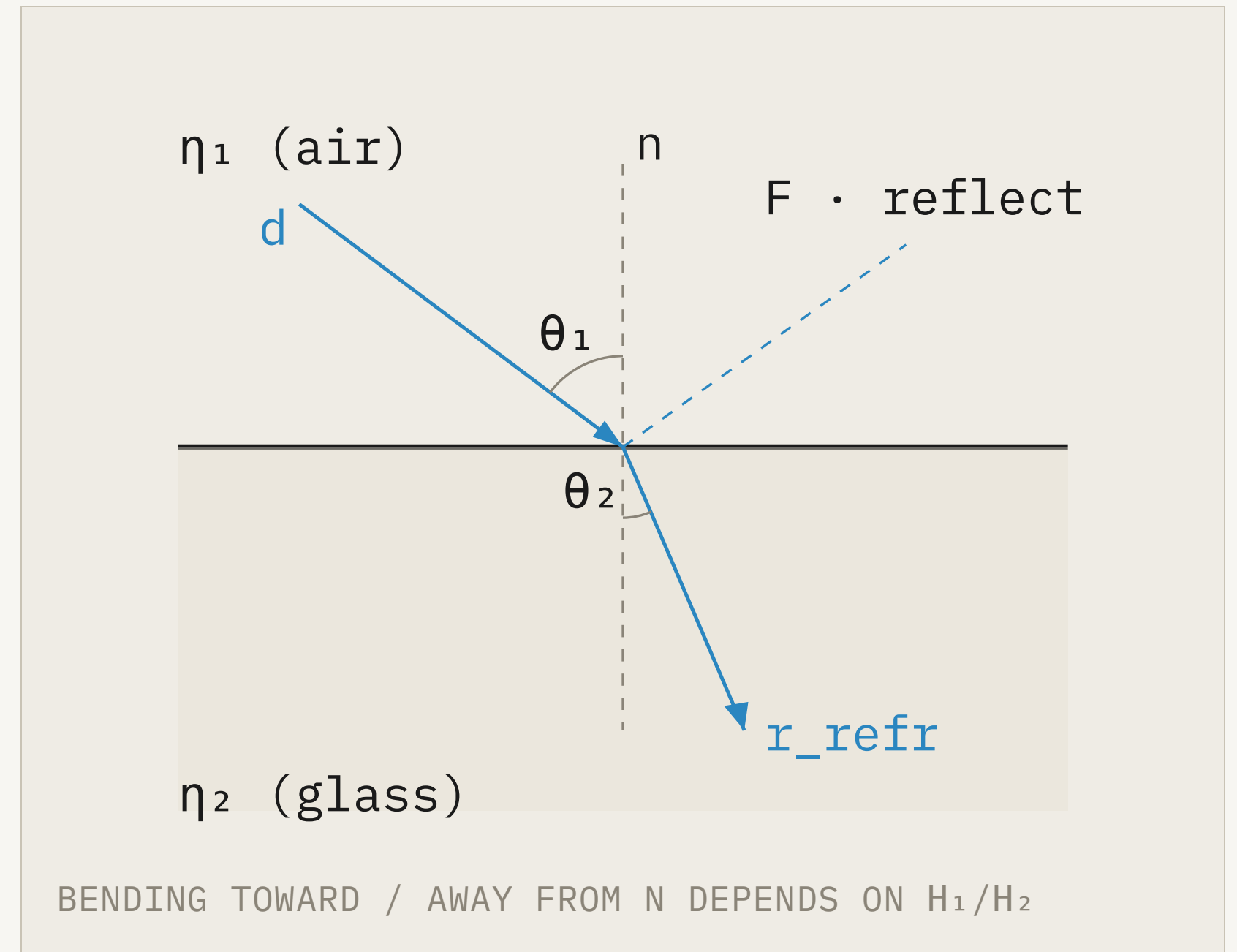
34 Refraction

Dielectric materials bend rays according to Snell's law and probabilistically reflect at grazing angles per Schlick's approximation.

$$\begin{aligned}\eta_1 \sin \theta_1 &= \eta_2 \sin \theta_2 \\ \eta &= \frac{\eta_1}{\eta_2}, \quad \cos \theta_1 = -\mathbf{d} \cdot \mathbf{n} \\ \mathbf{r}_\perp &= \eta (\mathbf{d} + \cos \theta_1 \mathbf{n}) \\ \mathbf{r}_\parallel &= -\sqrt{1 - \|\mathbf{r}_\perp\|^2} \mathbf{n} \\ \mathbf{r}_{\text{refracted}} &= \mathbf{r}_\perp + \mathbf{r}_\parallel\end{aligned}$$

Schlick's Fresnel. $r_0 = \left(\frac{1-\eta}{1+\eta}\right)^2$, $F = r_0 + (1 - r_0)(1 - \cos \theta_1)^5$. If $F > \xi \rightarrow$ reflect; else refract.

Total internal reflection triggers when $\|\mathbf{r}_\perp\|^2 > 1$.



35 Transparency

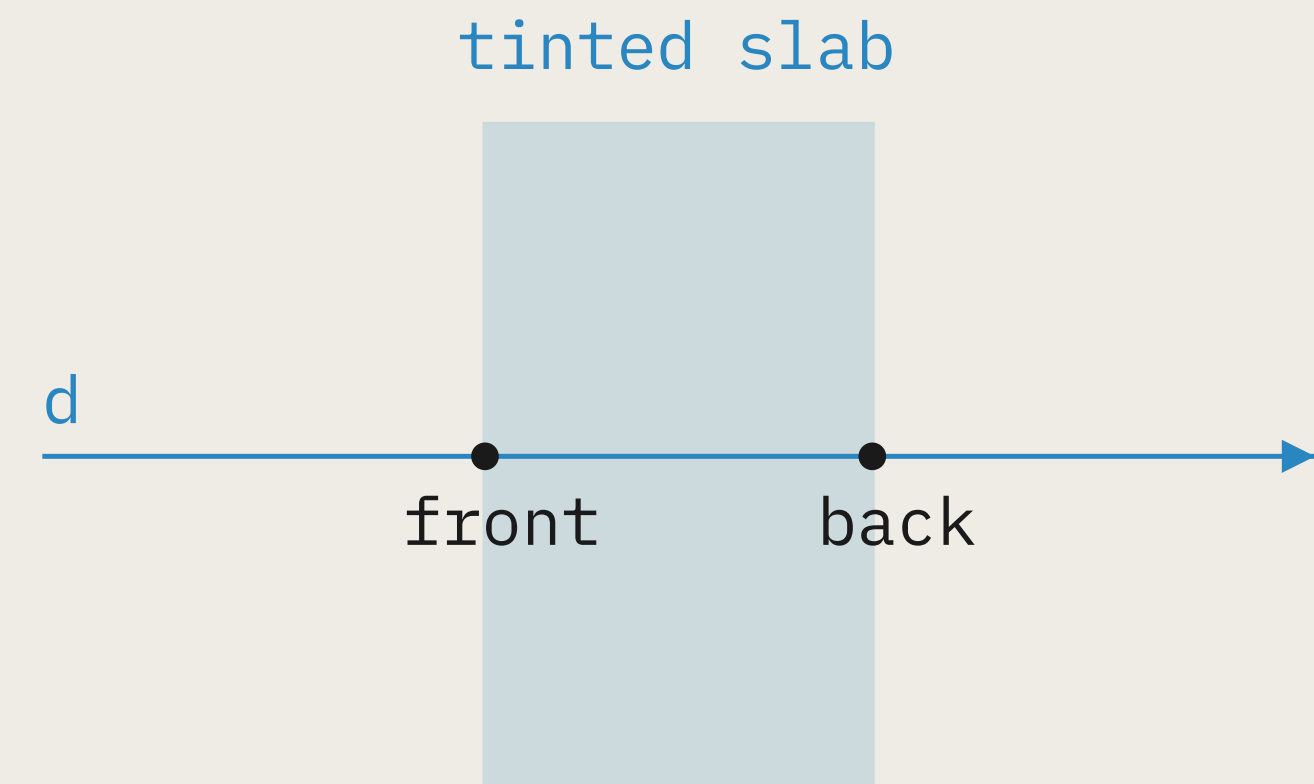
Like refraction but with $\eta = 1$: the ray continues unchanged through the surface. Useful for tinted glass without lensing.

$$\text{attenuation} = \begin{cases} \mathbf{C}_0 & \text{front face} \\ (1, 1, 1) & \text{back face} \end{cases}$$

$$\text{secondaryRay} = (P - \epsilon \mathbf{n}, \mathbf{d})$$

In shadow-ray traversal, each crossed transmissive surface multiplies its colour into the running `shadowFilter`, up to 8 bounces.

Coloured glass effects emerge from accumulated front-face attenuations along the path.



$$\text{attenuation} = \mathbf{C}_0 \cdot 1 \cdot \dots$$

NO BENDING — COLOUR ACCUMULATES ALONG THE RAY

36 Textured material

Reads an image texture using the UV coordinates emitted by each primitive's intersection routine.

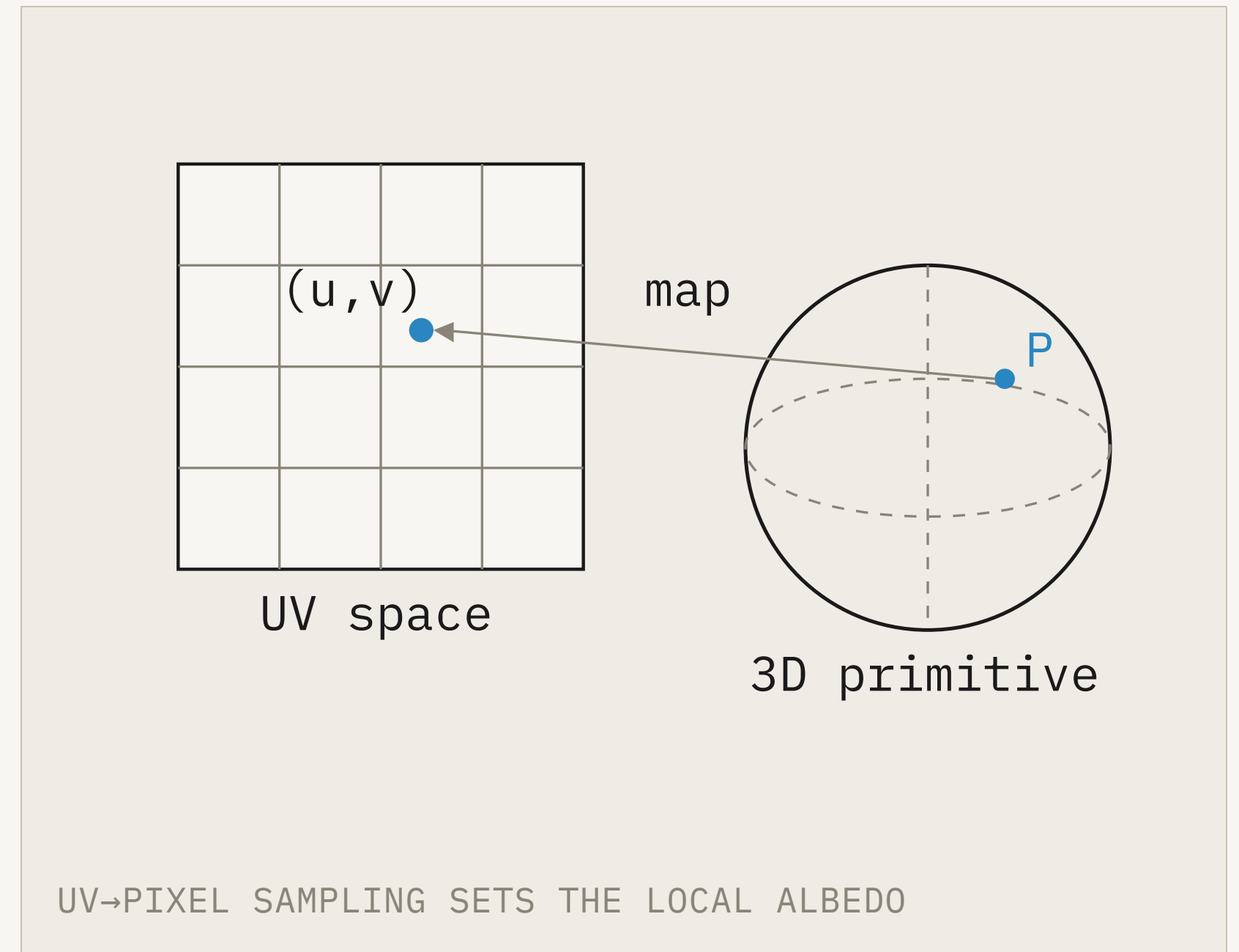
$$(u, v) \in [0, 1]^2$$

$$p_x = \lfloor u \cdot W \rfloor \bmod W, \quad p_y = \lfloor v \cdot H \rfloor \bmod H$$

$$\text{attenuation} = \text{tex}[p_x, p_y]$$

Each primitive defines its own UV map: spherical projection for spheres, tangent-frame projection for planes, barycentric interpolation for triangles, etc.

No secondary ray is produced — the texel acts as the local albedo for the lighting equation.



37 Procedural checkerboard

MATERIAL 7 / 9 • XOR PARITY

A purely procedural 3-D pattern based on the parity of the integer coordinates of the world-space hit point.

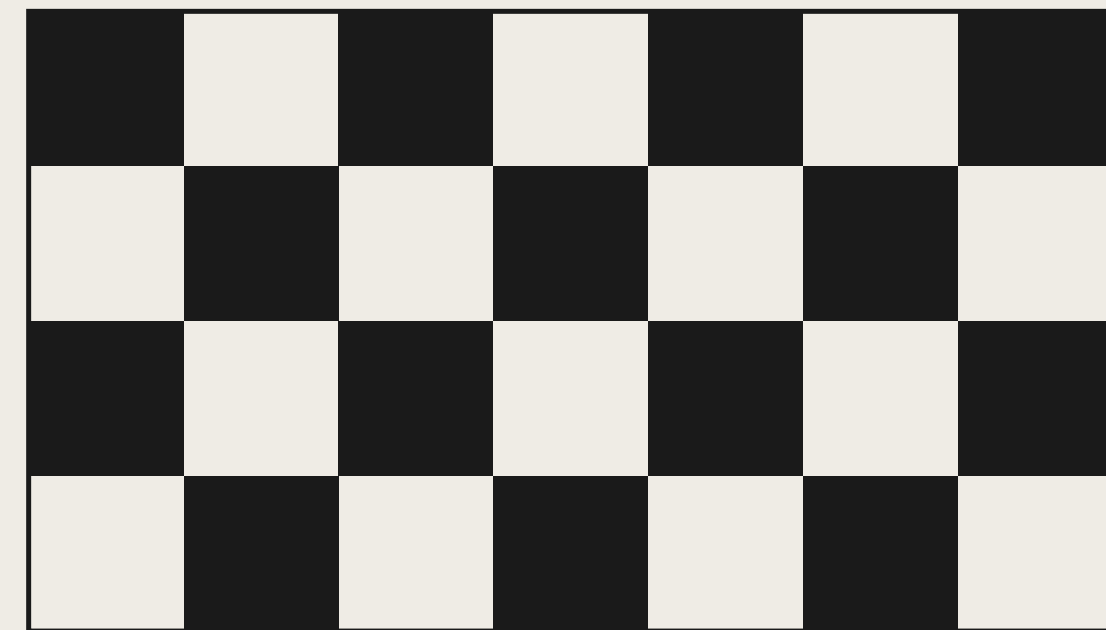
$$(i_x, i_y, i_z) = (\lfloor fx \rfloor, \lfloor fy \rfloor, \lfloor fz \rfloor)$$

$$p = (i_x \oplus i_y \oplus i_z) \& 1$$

$$\text{color} = \begin{cases} \mathbf{C}_A & \text{if } p = 1 \\ \mathbf{C}_B & \text{if } p = 0 \end{cases}$$

The XOR of three integers is odd iff an odd number of them is odd — this produces a coherent 3-D checkerboard in every axis-aligned plane.

f is the pattern frequency: large $f \rightarrow$ small tiles.



$$\text{parity} = (ix \oplus iy \oplus iz) \& 1$$

A 3-D PATTERN VISIBLE IN ANY CROSS-SECTION

38 Procedural marble

MATERIAL 8 / 9 • FBM + SINE

Marble veins are sine bands whose phase is perturbed by fractal Brownian motion — a sum of Perlin noise octaves.

$$\text{fBm}(P) = \sum_{i=0}^{N-1} \frac{1}{2^i} \text{perlin}(2^i P)$$

$$n = \sin(s P_z + \tau \text{fBm}(P))$$

$$t = \frac{n+1}{2}, \quad \text{color} = (1 - t) \mathbf{C}_A + t \mathbf{C}_B$$

s Vein frequency

τ Turbulence amplitude

N Number of fBm octaves (detail levels)

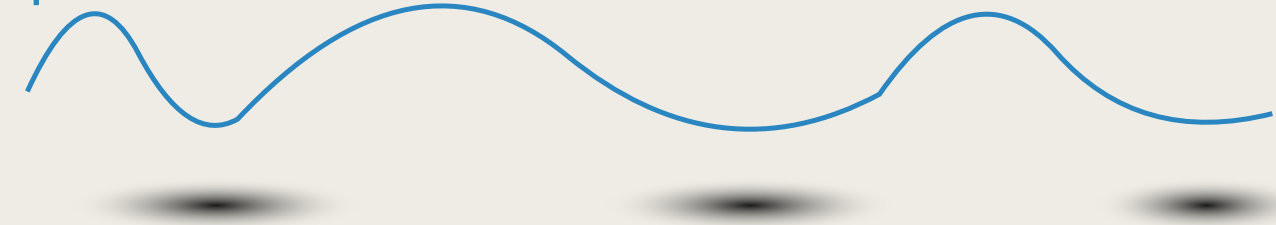
pure $\sin(s \cdot z)$



$\tau \cdot \text{fBm}(P)$



phase-modulated sine



SINE + FBM → IRREGULAR VEIN STRUCTURE

39 Normal map material

Perturbs the surface normal using an RGB texture encoding local normal directions in *tangent space*, then transforms back to world space through the TBN matrix.

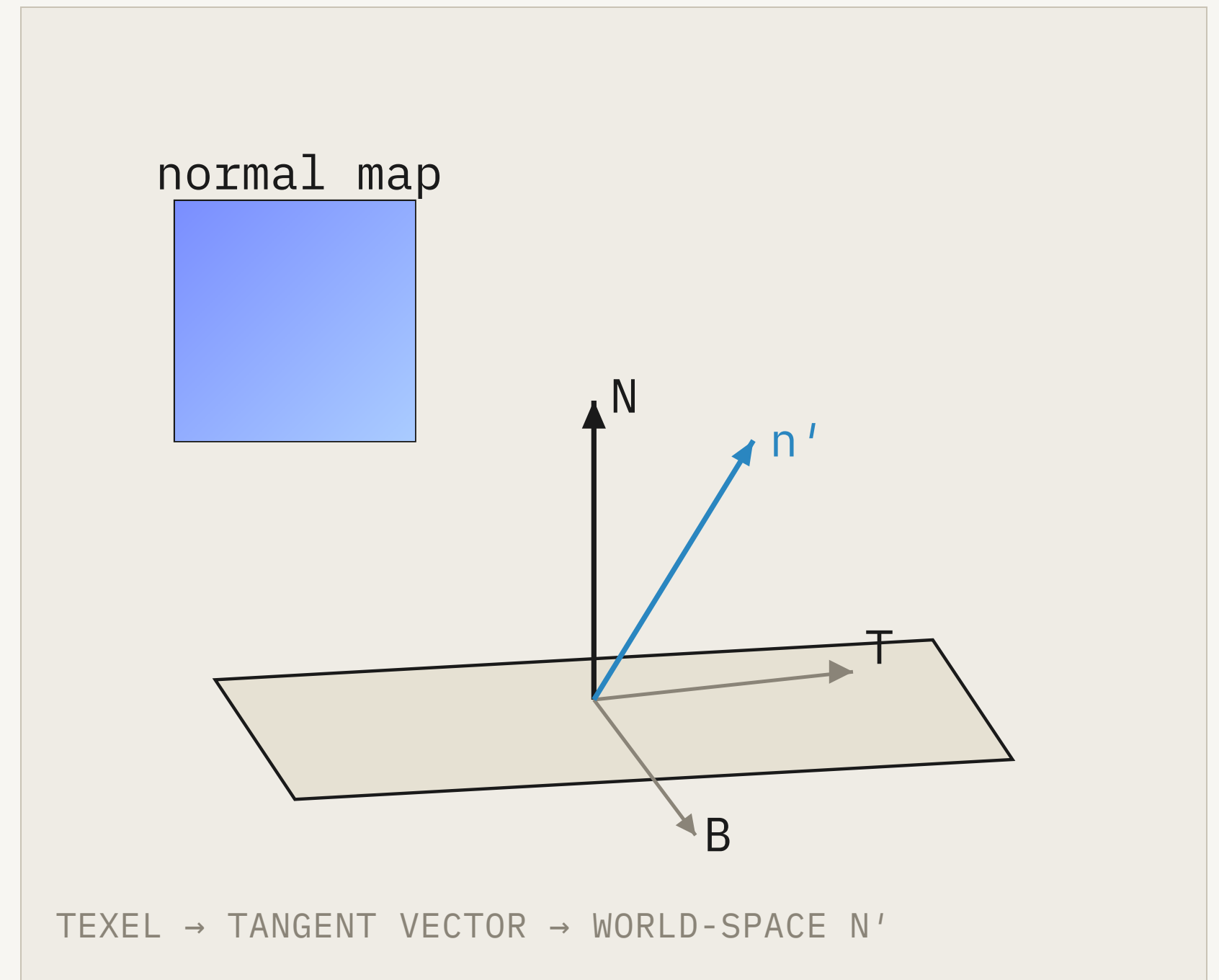
$$\mathbf{B} = \mathbf{N} \times \mathbf{T}, \quad \text{TBN} = [\mathbf{T} \ \mathbf{B} \ \mathbf{N}]$$

$$\mathbf{n}_{\text{tan}} = 2 \text{tex}(u, v) - 1$$

$$\mathbf{n}' = \widehat{\text{TBN} \mathbf{n}_{\text{tan}}}$$

The perturbed normal $\mathbf{n}'$ is returned through `ScatterResult.modifiedNormal` and used by every subsequent lighting term.

$\mathbf{T}$ and $\mathbf{B}$ are precomputed by the primitive's intersection routine and stored in the `HitRecord`.



BVH.
From $O(n)$ to $O(\log n)$.

41 Bounding volume hierarchy

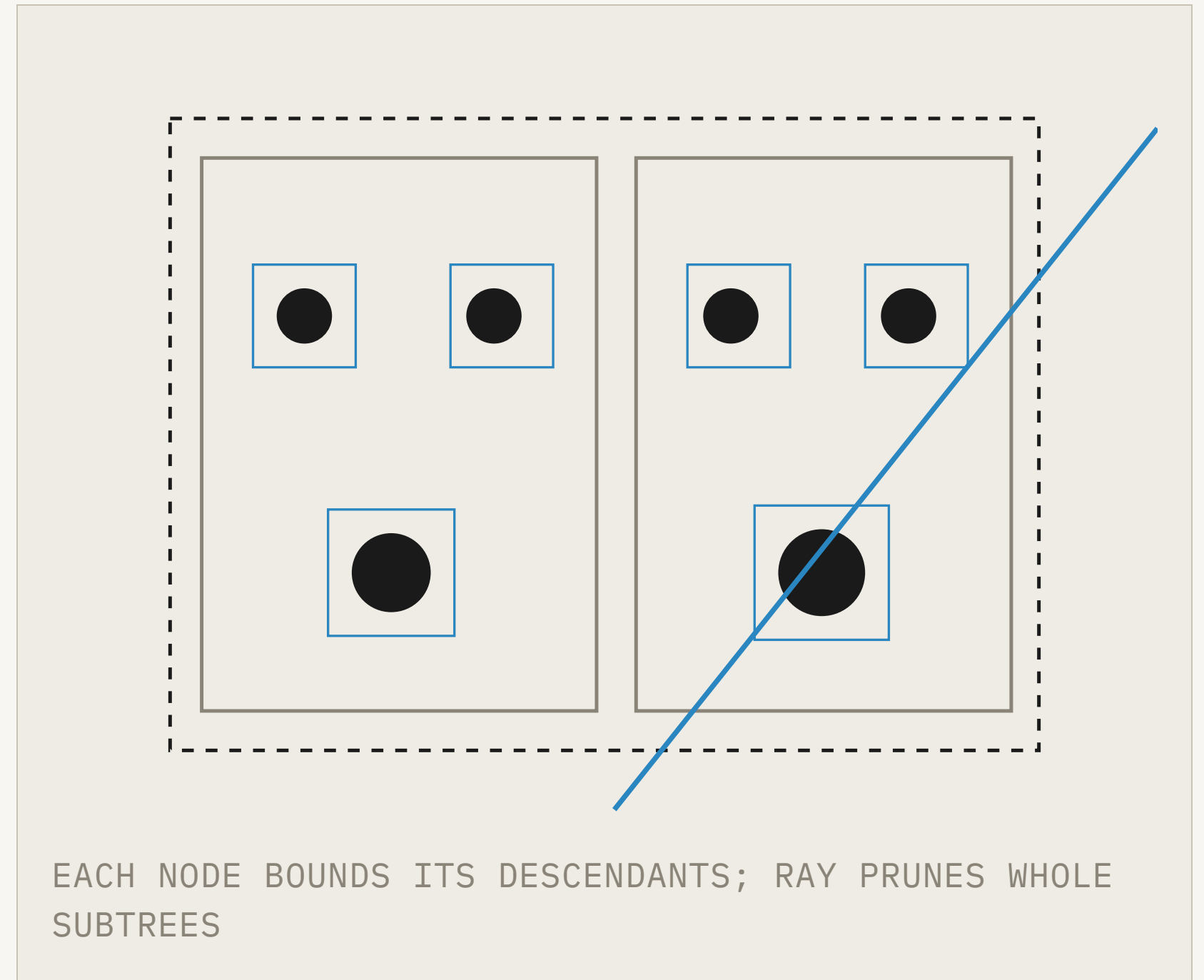
A binary tree of axis-aligned boxes built so a ray can prune entire branches with a single slab test, reducing scene intersection from $O(n)$ to $O(\log n)$.

$$\text{cost}(\text{split}) = 1 + \frac{\text{SA}(L)}{\text{SA}(\text{parent})} N_L + \frac{\text{SA}(R)}{\text{SA}(\text{parent})} N_R$$

At every node, we choose the axis and split position minimising the Surface-Area Heuristic cost — a model of the expected number of intersection tests.

```
function intersect(ray, node):  
    if not ray.hits(node.aabb): return ∅  
    if node.isLeaf: return closestHit(ray, node.prims)  
    h_L = intersect(ray, node.left)  
    h_R = intersect(ray, node.right)  
    return closer(h_L, h_R)
```

Unbounded primitives (infinite planes, infinite cylinders) bypass the BVH and are tested separately.



42 AABB — slab intersection

A box is the intersection of three orthogonal slabs. The ray crosses the box iff the three per-axis entry/exit intervals overlap.

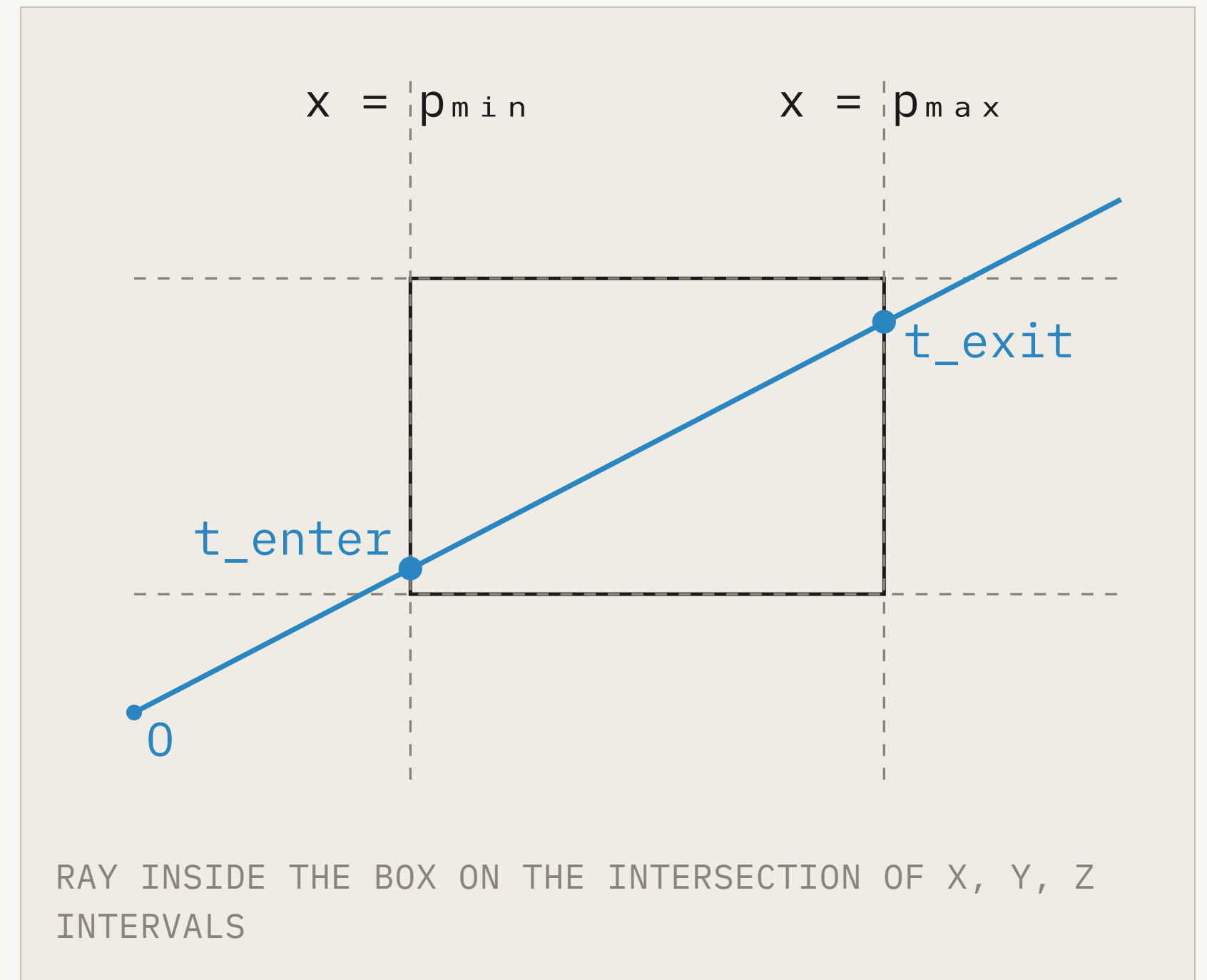
$$t_0^{(k)} = \frac{p_{\min}^{(k)} - O_k}{d_k}, \quad t_1^{(k)} = \frac{p_{\max}^{(k)} - O_k}{d_k}$$

$$t_{\text{enter}} = \max_k \min(t_0^{(k)}, t_1^{(k)})$$

$$t_{\text{exit}} = \min_k \max(t_0^{(k)}, t_1^{(k)})$$

Hit condition: $t_{\text{enter}} \leq t_{\text{exit}}$ and $t_{\text{exit}} > 0$.

If $d_k < 0$, swap $t_0^{(k)}$ and $t_1^{(k)}$. The whole test is branch-free and extremely cache-friendly.



From a parametric line to a complete image.

A handful of equations — quadratics, quartics, gradients, Snell's law — composed recursively through a BVH, are enough to reconstruct every shadow, every reflection, every vein of marble.